Reporte del proyecto: Algoritmos para el cálculo de todos los Testores Típicos de una matriz básica

Matemáticas Discretas — Semestre 202610

Integrantes:
Martín Enríquez, Paula Gallardo,
Eduardo Tapia

Profesor: Eduardo Alba
Universidad San Francisco de Quito (USFQ)

Fecha de culminación: 10 de noviembre de 2026

Índice

1. Fundamentos teóricos **2**

2. Metodología e implementación computacional **3**
2.1. Lenguaje, estructura y arquitectura . . . 3
2.2. Compilación y uso . . . 4

3. Ejercicio 1: generación de matrices y matriz básica **4**
3.1. Generación . . . 4
3.2. Reducción a matriz básica . . . 4
3.3. Resultado para una matriz de 100×10 . . . 5

4. Ejercicio 2: algoritmo YYC **5**
4.1. Ejercicio 2.1: matriz básica A . . . 5
4.2. El algoritmo YYC . . . 6
4.3. Ejercicio 2.2: ejecución manual de YYC sobre A . . . 6
4.4. Ejercicio 2.3: implementación computacional de YYC . . . 9
4.5. Ejercicio 2.4: resultados sobre la matriz A . . . 9

5. Ejercicio 3: algoritmo BT **10**
5.1. El algoritmo BT . . . 10
5.2. Ejercicio 3.1: ejecución manual de BT sobre A . . . 12
5.3. Ejercicio 3.2: implementación computacional de BT . . . 13
5.4. Ejercicio 3.3: resultados sobre la matriz A . . . 15

6. Ejercicio 4: construcción y análisis de la matriz B **16**
6.1. Ejercicio 4.1: construcción de la matriz B . . . 16
6.2. Ejercicio 4.2: aplicación del algoritmo BT sobre B . . . 17
6.3. Ejercicio 4.3: comparación entre A y B . . . 18

7. Ejercicio 5 **20**
7.1. Ejercicio 5.1: implementación de los operadores y generación de matrices 20
7.2. Ejercicio 5.2: resultados experimentales de YYC y BT 25
7.3. Ejercicio 5.3: análisis del desempeño de YYC y BT 26

1. Fundamentos teóricos

En Reconocimiento de Patrones con enfoque lógico-combinatorio, los objetos de estudio se describen mediante un conjunto de rasgos (características) $R = \{x_1, \ldots, x_n\}$. Comparando, rasgo a rasgo, objetos que pertenecen a clases distintas se obtiene una *matriz de diferencias* booleana: la entrada (i, j) vale 1 si el par de objetos i es distinguible por el rasgo x_j y 0 en caso contrario [3]. Toda la teoría de este proyecto trabaja sobre matrices booleanas de este tipo.

Definición 1 (Subfila y superfila)**.** Sean $p = (p_1, \ldots, p_n)$ y $t = (t_1, \ldots, t_n)$ dos filas de una matriz booleana. Decimos que p es *subfila* de t (y que t es *superfila* de p) si $p_j \leq t_j$ para todo j y $p \neq t$. Equivalentemente, el conjunto de columnas donde p tiene unos está estrictamente contenido en el de t.

Definición 2 (Fila básica y matriz básica)**.** Una fila de M es *básica* si ninguna otra fila de M es subfila suya. La *matriz básica* de M es la matriz formada exclusivamente por las filas básicas de M, sin repetición [4, 1]; se obtiene dejando una sola copia de cada fila repetida y eliminando todas las superfilas. Una matriz es *básica* si coincide con su matriz básica, es decir, si no tiene filas repetidas y ninguna de sus filas es subfila de otra.

Filas nulas. Una fila nula $p = (0, \ldots, 0)$ cumple trivialmente $p_j \leq t_j$ para todo j, así que es subfila de *toda* fila no nula. Por lo tanto, si M tiene una fila nula, su matriz básica es esa única fila nula. Esto es coherente con lo que sigue: ningún conjunto de rasgos cubre una fila nula, de modo que ni M ni su matriz básica tienen testores.

Definición 3 (Densidad)**.** La densidad de una matriz booleana M de $m \times n$ es la proporción de entradas iguales a 1:

$$\rho(M) = \frac{\#\{(i, j) : M_{ij} = 1\}}{m \cdot n}.$$

Definición 4 (Testor y testor típico)**.** Un subconjunto de rasgos $T \subseteq R$ es un *testor* de M si la submatriz $M|_T$ (formada por las columnas de T) no contiene filas nulas, es decir, si toda fila de M tiene al menos un 1 en alguna columna de T. T es un *testor típico* si es testor y ningún subconjunto propio de T es testor.

Definición 5 (Fila típica y conjunto compatible)**.** Sea $T \subseteq R$ y $x_j \in T$. Una fila r es *fila típica* de x_j respecto de T si $r_j = 1$ y $r_k = 0$ para todo $x_k \in T \setminus \{x_j\}$. T es un *conjunto compatible* de M si cada rasgo de T tiene al menos una fila típica, es decir, si $M|_T$ contiene, salvo permutación de filas, a la matriz identidad de orden $|T|$ [1].

Proposición 1. *T es testor típico de M si y sólo si T es testor y conjunto compatible de M.*

Demostración. Ser testor es una propiedad monótona: si T es testor, todo superconjunto también lo es. Por tanto, T es típico si y sólo si $T \setminus \{x_j\}$ no es testor para cada $x_j \in T$. Que $T \setminus \{x_j\}$ no sea testor significa que existe una fila r que tiene sólo ceros en $T \setminus \{x_j\}$; como T sí es testor, $r_j = 1$. Esa fila es precisamente una fila típica de x_j. □

Proposición 2. *Una matriz M y su matriz básica BM tienen exactamente los mismos testores y, por lo tanto, los mismos testores típicos.*

Demostración. Si t es superfila de p, todo T que cubre a p cubre a t; luego eliminar t (o una copia repetida) no cambia el conjunto de testores. Como los testores típicos son los elementos minimales del conjunto de testores, tampoco cambian. El argumento vale también cuando p es una fila nula: ningún T la cubre, así que ni M ni BM tienen testores. □

Por eso la fila nula *no* debe descartarse. Si se eliminara, el resultado dejaría de ser equivalente a M: con $M = \left(\begin{smallmatrix} 1 & 1 & 0 \\ 0 & 0 & 0 \\ 0 & 1 & 1 \end{smallmatrix}\right)$, que no tiene testores, quitar la fila nula deja una matriz cuyos testores típicos son $\{x_2\}$ y $\{x_1, x_3\}$.

La Proposición 2 justifica que los algoritmos trabajen siempre sobre la matriz básica, que es (en general) mucho más pequeña.

2. Metodología e implementación computacional

2.1. Lenguaje, estructura y arquitectura

La implementación se realizó en C++17 y se compila con CMake. El código se organizó siguiendo una *arquitectura limpia* de cuatro capas, cada una compilada como una librería independiente. El dominio y los casos de uso no conocen los detalles de archivos o de consola; la composición de dependencias concretas se realiza en el punto de entrada:

Capa	Responsabilidad
domain	Conceptos matemáticos puros: `ColumnSet` (conjunto de columnas como mapa de bits), `BooleanMatrix`, reducción a matriz básica, definiciones de testor, fila típica y conjunto compatible, y los algoritmos `YycSolver` y `BtSolver`. No realiza entrada/salida.
application	Casos de uso: generar matrices, buscar una matriz básica que cumpla restricciones, ejecutar YYC midiendo el tiempo acumulado por fila y ejecutar BT midiendo su tiempo total. Define *puertos* (interfaces) para el azar, el reloj y la notificación de progreso.
infrastructure	Implementaciones concretas de los puertos: generador Mersenne Twister de 64 bits con semilla, reloj monótono, lectura/escritura de matrices y exportación de tablas a CSV y LaTeX.
presentation	Interfaz de usuario: menú interactivo y subcomandos de línea de comandos.

Representación. Cada fila de la matriz y cada conjunto de rasgos se representa con un `ColumnSet`: un arreglo fijo de ocho palabras de 64 bits (hasta 512 columnas). Las operaciones "T cubre a la fila r" ($T \cap r \neq \emptyset$) y "$T \cap r$ tiene exactamente un elemento" se resuelven con operaciones `AND` y conteo de bits sobre palabras completas. El almacenamiento fijo permite crear los conjuntos temporales del ciclo interno sin reservar memoria dinámica para cada conjunto; las listas de conjuntos sí usan memoria dinámica.

Seguridad y robustez.

- Toda entrada numérica se valida de forma estricta (rango y cadena completa); una entrada inválida vuelve a preguntarse en lugar de dejar el flujo en error.
- Los archivos de matriz se validan carácter por carácter (sólo `0`, `1` y separadores) y se informa la línea del error; se limita el tamaño del archivo (16 MiB) y de la matriz (10^5 filas, 512 columnas).
- YYC tiene una cota configurable sobre $|\Psi^*|$ (2 millones de conjuntos por defecto). El consumo real también incluye las familias intermedias, sus capacidades reservadas y las estructuras auxiliares; la cota no equivale a un límite estricto de memoria en bytes.

- No se sobrescribe ningún archivo existente sin autorización explícita (`-forzar` o confirmación en el menú).
- Se compila con todas las advertencias activadas (`-Wall -Wextra -Wconversion...`) sin ninguna advertencia, y con protecciones del compilador (`-fstack-protector-strong`, `_FORTIFY_SOURCE`, `_GLIBCXX_ASSERTIONS`).

Verificación. El proyecto incluye 82 pruebas automatizadas. La más importante compara el resultado de YYC con un *oráculo exhaustivo* independiente (recorre los 2^n subconjuntos de columnas y aplica directamente la definición de testor típico) sobre 400 matrices básicas aleatorias de hasta 12 columnas, y además comprueba que, después de cada fila, Ψ^* coincide exactamente con los testores típicos de las filas procesadas. Las pruebas de BT se describen en la Sección 5.3.

Reproducibilidad. Toda generación aleatoria usa una semilla que el programa informa. El bit aleatorio se obtiene como $u < p$ con $u \in [0, 1)$ formado por los 53 bits altos de `mt19937_64`, cuya secuencia está fijada por el estándar de C++; así, la misma semilla produce la misma matriz con cualquier compilador. El script `scripts/generar_resultados.sh` regenera todos los datos y tablas de este reporte.

2.2. Compilación y uso

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build
ctest --test-dir build                  # pruebas
./build/testores                        # menú interactivo
./build/testores ayuda                  # subcomandos disponibles
```

Las mediciones incluidas se obtuvieron en Windows 11 (AMD64), con procesador 12th Gen Intel(R) Core(TM) i7-1265U, compilador GNU 16.1.0, C++17 y configuración Release. Se utilizó un reloj monótono de alta resolución. Las tablas de A y B usan 10 000 ejecuciones; las tablas de operadores usan 9 ejecuciones medidas precedidas de un calentamiento. Los tiempos dependen de la carga del equipo y no son constantes matemáticas.

3. Ejercicio 1: generación de matrices y matriz básica

Enunciado. Desarrollar un programa que genere matrices booleanas aleatorias de dimensiones dadas (al menos hasta 100×10), que transforme cada matriz en su matriz básica y calcule la densidad de esta última.

3.1. Generación

El usuario indica el número de filas $m \in [1, 10^5]$, de columnas $n \in [1, 512]$, la probabilidad p de que una entrada sea 1 (por defecto $p = 0.5$) y, opcionalmente, una semilla. Cada entrada se genera de forma independiente como una variable de Bernoulli de parámetro p.

3.2. Reducción a matriz básica

El procedimiento `reduceToBasic` aplica la definición en dos pasos y, además de la matriz básica, devuelve para cada fila original la razón por la que se conservó o eliminó:

1. Recorre las filas en orden y descarta las que son idénticas a una fila anterior ya aceptada (se conserva la primera aparición).

2. Entre las filas restantes, conserva exactamente las *minimales*: aquellas que no tienen ninguna subfila. Cada superfila eliminada se justifica con una subfila suya que sí permanece en la matriz básica (existe por transitividad de la relación de subfila, que es un orden parcial estricto sobre filas distintas).

Las filas nulas no necesitan un tratamiento especial: como una fila nula es subfila de toda fila no nula, el paso 2 elimina todas las demás y la matriz básica queda formada por una sola fila nula (de densidad 0), tal como exige la definición. En ese caso el programa avisa que la matriz no tiene testores, y YYC y BT devuelven el conjunto vacío. El orden relativo de las filas que permanecen no cambia y el número de columnas tampoco. El costo es $O(m^2\lceil n/64\rceil)$. Finalmente la densidad se calcula contando los unos con conteo de bits por palabra.

3.3. Resultado para una matriz de 100×10

Con `testores generar -filas 100 -columnas 10 -semilla 2026` se obtuvo una matriz aleatoria de densidad 0.51, sin filas nulas. La reducción eliminó 5 filas repetidas y 82 superfilas, y produjo la siguiente matriz básica de 13×10 (archivo `data/ej1_basica_100x10.txt`):

$$BM = \begin{pmatrix} 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{pmatrix}, \qquad \rho(BM) = \frac{39}{13 \cdot 10} = 0.30.$$

En este ejemplo la densidad disminuye (0.30 frente a 0.51). No es una propiedad general: una superfila tiene más unos que la subfila que justifica su eliminación, pero puede tener menos unos que el promedio de las otras filas incomparables. Por ejemplo, las filas 10000, 11000 y 01111 tienen densidad 7/15; eliminar 11000 deja una básica de densidad 5/10, que es mayor. Las filas con un único 1 (la de x_2 y la de x_7) obligan a que esos rasgos pertenezcan a todo testor de la matriz.

4. Ejercicio 2: algoritmo YYC

4.1. Ejercicio 2.1: matriz básica A

Enunciado. Generar una matriz básica A con al menos siete columnas y densidad mayor o igual a 0.4.

Para obtener una básica que cumpla simultáneamente las restricciones, se implementó el caso de uso `searchBasicMatrix`: genera matrices aleatorias con una semilla fija hasta que la matriz básica cumpla las restricciones (densidad mínima, rango de filas y que ninguna columna quede nula). Se pidió una matriz básica de 7 columnas y 6 filas (tamaño adecuado para la ejecución manual):

```
testores buscar --columnas 7 --filas 8 --densidad-min 0.4 \
   --min-filas-basica 6 --max-filas-basica 6 \
   --semilla 2026 --salida data/matriz_A.txt
```

El primer intento ya cumplió las condiciones. La matriz aleatoria generada (izquierda) tiene densidad $31/56 \approx 0.5536$; la fila r_2 es superfila de $r_5 = (0,0,1,1,0,1,1)$ y r_6 también es superfila de r_5, por lo que ambas se eliminan. El resultado es la matriz básica A (derecha):

$$M = \begin{pmatrix} 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 \\ 1 & 0 & 0 & 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 & 1 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 0 & 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 1 \end{pmatrix} \longrightarrow \quad A = \begin{array}{c} \begin{matrix} x_1 & x_2 & x_3 & x_4 & x_5 & x_6 & x_7 \end{matrix} \\ \begin{pmatrix} 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 1 \end{pmatrix} \begin{matrix} r_1 \\ r_2 \\ r_3 \\ r_4 \\ r_5 \\ r_6 \end{matrix} \end{array}$$

A tiene 20 unos en $6 \cdot 7 = 42$ entradas, luego $\rho(A) = 20/42 \approx \mathbf{0.4762} \geq 0.4$, y tiene 7 columnas, todas con al menos un 1. Se verifica que es básica: no hay filas nulas ni repetidas, y para cada par de filas distintas cada una tiene un 1 donde la otra tiene 0.

4.2. El algoritmo YYC

YYC [1] es un algoritmo *incremental*: procesa las filas de la matriz básica una por una y mantiene el conjunto Ψ^* de los testores típicos de la submatriz formada por las filas ya procesadas.

1. **Inicialización.** $\Psi^* \leftarrow \{\{x_j\} : r_1[j] = 1\}$.
2. **Para cada fila** r_k, $k = 2, \ldots, m$: $\Psi_{aux} \leftarrow \emptyset$ y, para cada $\tau \in \Psi^*$:
   - si $\tau \cap r_k \neq \emptyset$ (τ ya cubre la fila), τ pasa a Ψ_{aux} sin cambios;
   - si no, para cada rasgo x con $r_k[x] = 1$, si $\tau \cup \{x\}$ es un conjunto compatible de las filas $r_1, \ldots, r_k$, se agrega a Ψ_{aux}.

   Al terminar, $\Psi^* \leftarrow \Psi_{aux}$.
3. Al procesar la última fila, Ψ^* es el conjunto de todos los testores típicos de la matriz.

Por qué funciona. Por inducción, supongamos que Ψ^* contiene los testores típicos de las filas $r_1, \ldots, r_{k-1}$. Todo elemento de Ψ_{aux} cubre r_k y las filas anteriores, por lo que es testor de las primeras k filas; además se exige que sea compatible, así que por la Proposición 1 es típico. Recíprocamente, si T es típico para las primeras k filas, o bien ya era típico antes (y cubre r_k), o bien $T = \tau \cup \{x\}$ donde r_k es la fila típica de x y τ era típico para las filas anteriores. Por último, no se generan duplicados: si $\tau_1 \cup \{x\} = \tau_2 \cup \{y\}$ con $x \neq y$, entonces $y \in \tau_1$, pero $y \in r_k$ y τ_1 no corta a r_k, lo cual es una contradicción.

Observación útil para el cálculo manual. Al extender un τ que no corta a r_k con un $x \in r_k$, la propia fila r_k es fila típica de x en $\tau \cup \{x\}$ (pues r_k tiene ceros en todo τ). Por lo tanto basta revisar que *cada rasgo de τ conserve alguna fila típica*, es decir, que tenga una fila donde aparece sin x ni otros rasgos de τ. Agregar x sólo puede "romper" filas típicas de τ que tengan un 1 en la columna x.

4.3. Ejercicio 2.2: ejecución manual de YYC sobre A

Escribimos cada fila de A como el conjunto de rasgos donde tiene un 1:

$$r_1 = \{x_1, x_3, x_5, x_7\}, \quad r_2 = \{x_1, x_5, x_6\}, \quad r_3 = \{x_1, x_2, x_4\},$$
$$r_4 = \{x_3, x_4, x_6, x_7\}, \quad r_5 = \{x_1, x_4, x_6\}, \quad r_6 = \{x_1, x_4, x_7\}.$$

En cada paso se recorre Ψ^* en el orden en que fue construido. Para cada candidato $\tau \cup \{x\}$ se escribe la intersección de cada fila procesada con el candidato; una intersección con un único

elemento $\{x_j\}$ es una fila típica de x_j. El símbolo ✓ indica que el candidato es compatible y se agrega; × indica que se descarta y se señala el rasgo que se queda sin fila típica.

Paso 1: fila $r_1 = \{x_1, x_3, x_5, x_7\}$

Inicialización: cada rasgo presente en r_1 forma un conjunto unitario.

$$\Psi^* = \{\{x_1\},\ \{x_3\},\ \{x_5\},\ \{x_7\}\} \qquad |\Psi^*| = 4.$$

Paso 2: fila $r_2 = \{x_1, x_5, x_6\}$

- $\tau = \{x_1\}$: $\tau \cap r_2 = \{x_1\} \neq \emptyset \Rightarrow$ se conserva.
- $\tau = \{x_3\}$: no cubre r_2; se prueba con $x \in \{x_1, x_5, x_6\}$:
  - $\{x_1, x_3\}$: $r_1 \to \{x_1, x_3\}$, $r_2 \to \{x_1\}$. x_3 sólo aparece en r_1 junto a x_1: × (x_3 sin fila típica).
  - $\{x_3, x_5\}$: $r_1 \to \{x_3, x_5\}$, $r_2 \to \{x_5\}$: × (x_3 sin fila típica).
  - $\{x_3, x_6\}$: $r_1 \to \{x_3\}$, $r_2 \to \{x_6\}$: ✓.
- $\tau = \{x_5\}$: $\tau \cap r_2 = \{x_5\} \Rightarrow$ se conserva.
- $\tau = \{x_7\}$: no cubre r_2:
  - $\{x_1, x_7\}$: $r_1 \to \{x_1, x_7\}$, $r_2 \to \{x_1\}$: × (x_7 sin fila típica).
  - $\{x_5, x_7\}$: $r_1 \to \{x_5, x_7\}$, $r_2 \to \{x_5\}$: × (x_7 sin fila típica).
  - $\{x_6, x_7\}$: $r_1 \to \{x_7\}$, $r_2 \to \{x_6\}$: ✓.

$$\Psi^* = \{\{x_1\},\ \{x_3, x_6\},\ \{x_5\},\ \{x_6, x_7\}\} \qquad |\Psi^*| = 4.$$

Paso 3: fila $r_3 = \{x_1, x_2, x_4\}$

- $\tau = \{x_1\}$: cubre $r_3 \Rightarrow$ se conserva.
- $\tau = \{x_3, x_6\}$: no cubre r_3:
  - $\{x_1, x_3, x_6\}$: $r_1 \to \{x_1, x_3\}$, $r_2 \to \{x_1, x_6\}$, $r_3 \to \{x_1\}$: × (x_3 y x_6 pierden sus filas típicas r_1 y r_2, que contienen a x_1).
  - $\{x_2, x_3, x_6\}$: $r_1 \to \{x_3\}$, $r_2 \to \{x_6\}$, $r_3 \to \{x_2\}$: ✓.
  - $\{x_3, x_4, x_6\}$: $r_1 \to \{x_3\}$, $r_2 \to \{x_6\}$, $r_3 \to \{x_4\}$: ✓.
- $\tau = \{x_5\}$: no cubre r_3:
  - $\{x_1, x_5\}$: $r_1 \to \{x_1, x_5\}$, $r_2 \to \{x_1, x_5\}$, $r_3 \to \{x_1\}$: × (x_5 sin fila típica).
  - $\{x_2, x_5\}$: $r_1 \to \{x_5\}$, $r_2 \to \{x_5\}$, $r_3 \to \{x_2\}$: ✓.
  - $\{x_4, x_5\}$: $r_1 \to \{x_5\}$, $r_2 \to \{x_5\}$, $r_3 \to \{x_4\}$: ✓.
- $\tau = \{x_6, x_7\}$: no cubre r_3:
  - $\{x_1, x_6, x_7\}$: $r_1 \to \{x_1, x_7\}$, $r_2 \to \{x_1, x_6\}$, $r_3 \to \{x_1\}$: × (x_6 y x_7 sin fila típica).
  - $\{x_2, x_6, x_7\}$: $r_1 \to \{x_7\}$, $r_2 \to \{x_6\}$, $r_3 \to \{x_2\}$: ✓.
  - $\{x_4, x_6, x_7\}$: $r_1 \to \{x_7\}$, $r_2 \to \{x_6\}$, $r_3 \to \{x_4\}$: ✓.

$$\Psi^* = \{\{x_1\},\ \{x_2, x_3, x_6\},\ \{x_3, x_4, x_6\},\ \{x_2, x_5\},\ \{x_4, x_5\},\ \{x_2, x_6, x_7\},\ \{x_4, x_6, x_7\}\} \qquad |\Psi^*| = 7.$$

Paso 4: fila $r_4 = \{x_3, x_4, x_6, x_7\}$

- $\tau = \{x_1\}$: no cubre r_4. Respecto de $\{x_1\}$, las filas r_1, r_2, r_3 son filas típicas de x_1; al agregar x basta que alguna de ellas no contenga a x:
  - $\{x_1, x_3\}$: $r_2 \to \{x_1\}$, $r_4 \to \{x_3\}$: ✓.
  - $\{x_1, x_4\}$: $r_1 \to \{x_1\}$, $r_4 \to \{x_4\}$: ✓.
  - $\{x_1, x_6\}$: $r_1 \to \{x_1\}$, $r_4 \to \{x_6\}$: ✓.
  - $\{x_1, x_7\}$: $r_2 \to \{x_1\}$, $r_4 \to \{x_7\}$: ✓.
- $\tau = \{x_2, x_3, x_6\}$: $\tau \cap r_4 = \{x_3, x_6\} \Rightarrow$ se conserva.
- $\tau = \{x_3, x_4, x_6\}$: $\tau \cap r_4 = \{x_3, x_4, x_6\} \Rightarrow$ se conserva.

- $\tau = \{x_2, x_5\}$: no cubre r_4 (sus filas típicas: r_3 para x_2; r_1, r_2 para x_5):
  - $\{x_2, x_3, x_5\}$: $r_2 \to \{x_5\}$, $r_3 \to \{x_2\}$, $r_4 \to \{x_3\}$: ✓.
  - $\{x_2, x_4, x_5\}$: $r_3 \to \{x_2, x_4\}$ y x_2 no aparece en otra fila: × (x_2 sin fila típica).
  - $\{x_2, x_5, x_6\}$: $r_1 \to \{x_5\}$, $r_3 \to \{x_2\}$, $r_4 \to \{x_6\}$: ✓.
  - $\{x_2, x_5, x_7\}$: $r_2 \to \{x_5\}$, $r_3 \to \{x_2\}$, $r_4 \to \{x_7\}$: ✓.
- $\tau = \{x_4, x_5\}$: $\tau \cap r_4 = \{x_4\} \Rightarrow$ se conserva.
- $\tau = \{x_2, x_6, x_7\}$: $\tau \cap r_4 = \{x_6, x_7\} \Rightarrow$ se conserva.
- $\tau = \{x_4, x_6, x_7\}$: $\tau \cap r_4 = \{x_4, x_6, x_7\} \Rightarrow$ se conserva.

$$\Psi^* = \{\{x_1, x_3\},\ \{x_1, x_4\},\ \{x_1, x_6\},\ \{x_1, x_7\},\ \{x_2, x_3, x_6\},\ \{x_3, x_4, x_6\},\ \{x_2, x_3, x_5\},\ \{x_2, x_5, x_6\},\ \{x_2, x_5, x_7\},\ \{x_4, x_5\},\ \{x_2, x_6, x_7\},\ \{x_4, x_6, x_7\}\}$$
$$|\Psi^*| = 12.$$

Paso 5: fila $r_5 = \{x_1, x_4, x_6\}$

Diez de los doce conjuntos cortan a r_5 y se conservan: los cuatro que contienen x_1, los que contienen x_6 ($\{x_2, x_3, x_6\}$, $\{x_3, x_4, x_6\}$, $\{x_2, x_5, x_6\}$, $\{x_2, x_6, x_7\}$, $\{x_4, x_6, x_7\}$) y $\{x_4, x_5\}$. Sólo hay que extender dos:

- $\tau = \{x_2, x_3, x_5\}$: la única fila donde aparece x_2 es $r_3 = \{x_1, x_2, x_4\}$.
  - $\{x_1, x_2, x_3, x_5\}$: $r_3 \to \{x_1, x_2\}$: × (x_2 sin fila típica).
  - $\{x_2, x_3, x_4, x_5\}$: $r_3 \to \{x_2, x_4\}$: × (x_2 sin fila típica).
  - $\{x_2, x_3, x_5, x_6\}$: $r_1 \to \{x_3, x_5\}$, $r_2 \to \{x_5, x_6\}$, $r_4 \to \{x_3, x_6\}$: × (x_3 y x_5 sin fila típica).
- $\tau = \{x_2, x_5, x_7\}$:
  - $\{x_1, x_2, x_5, x_7\}$ y $\{x_2, x_4, x_5, x_7\}$: igual que antes, r_3 deja de ser típica de x_2: ×.
  - $\{x_2, x_5, x_6, x_7\}$: $r_1 \to \{x_5, x_7\}$, $r_2 \to \{x_5, x_6\}$: × (x_5 sin fila típica).

Ninguna extensión es compatible, así que $\{x_2, x_3, x_5\}$ y $\{x_2, x_5, x_7\}$ desaparecen:

$$\Psi^* = \{\{x_1, x_3\},\ \{x_1, x_4\},\ \{x_1, x_6\},\ \{x_1, x_7\},\ \{x_2, x_3, x_6\},\ \{x_3, x_4, x_6\},\ \{x_2, x_5, x_6\},\ \{x_4, x_5\},\ \{x_2, x_6, x_7\},\ \{x_4, x_6, x_7\}\} \qquad |\Psi^*| = 10.$$

Paso 6: fila $r_6 = \{x_1, x_4, x_7\}$

Cortan a r_6 y se conservan: $\{x_1, x_3\}$, $\{x_1, x_4\}$, $\{x_1, x_6\}$, $\{x_1, x_7\}$, $\{x_3, x_4, x_6\}$, $\{x_4, x_5\}$, $\{x_2, x_6, x_7\}$ y $\{x_4, x_6, x_7\}$. No cortan a r_6:

- $\tau = \{x_2, x_3, x_6\}$: con $x = x_1$ o $x = x_4$ la fila r_3 deja de ser típica de x_2 (×); con $x = x_7$, $\{x_2, x_3, x_6, x_7\}$: $r_1 \to \{x_3, x_7\}$, $r_4 \to \{x_3, x_6, x_7\}$, así que x_3 queda sin fila típica (×).
- $\tau = \{x_2, x_5, x_6\}$: con $x = x_1$ o $x = x_4$, x_2 pierde r_3 (×); con $x = x_7$, $\{x_2, x_5, x_6, x_7\}$: $r_1 \to \{x_5, x_7\}$, $r_2 \to \{x_5, x_6\}$, así que x_5 queda sin fila típica (×).

$$\boxed{\Psi^* = \{\{x_1, x_3\},\ \{x_1, x_4\},\ \{x_1, x_6\},\ \{x_1, x_7\},\ \{x_4, x_5\},\ \{x_2, x_6, x_7\},\ \{x_3, x_4, x_6\},\ \{x_4, x_6, x_7\}\}} \qquad |\Psi^*| = 8.$$

Comprobación. La Tabla 1 verifica, para cada uno de los 8 testores típicos, que es testor (ninguna fila queda con intersección vacía) y que cada rasgo tiene una fila típica.

Observaciones de la ejecución:

- $|\Psi^*|$ no es monótono: pasa por $4, 4, 7, 12, 10, 8$. Las filas r_5 y r_6 eliminan candidatos, porque ningún conjunto que no las cubre admite una extensión compatible.
- Las extensiones se descartan casi siempre por la misma razón: el único rasgo x disponible en la nueva fila aparece también en la única fila típica de algún rasgo de τ (por ejemplo, x_2 sólo tiene a r_3, que también contiene a x_1 y x_4).
- En total se evaluaron $6+9+8+6+6 = 35$ candidatos y $2+6+7 = 15$ fueron compatibles; nunca se examinó un subconjunto que no fuera extensión directa de un elemento de Ψ^*, mientras que una búsqueda exhaustiva examinaría $2^7 - 1 = 127$ subconjuntos.

T	$r_1 \cap T$	$r_2 \cap T$	$r_3 \cap T$	$r_4 \cap T$	$r_5 \cap T$	$r_6 \cap T$	Filas típicas
$\{x_1, x_3\}$	x_1x_3	x_1	x_1	x_3	x_1	x_1	x_1:r_2, x_3:r_4
$\{x_1, x_4\}$	x_1	x_1	x_1x_4	x_4	x_1x_4	x_1x_4	x_1:r_1, x_4:r_4
$\{x_1, x_6\}$	x_1	x_1x_6	x_1	x_6	x_1x_6	x_1	x_1:r_1, x_6:r_4
$\{x_1, x_7\}$	x_1x_7	x_1	x_1	x_7	x_1	x_1x_7	x_1:r_2, x_7:r_4
$\{x_4, x_5\}$	x_5	x_5	x_4	x_4	x_4	x_4	x_4:r_3, x_5:r_1
$\{x_2, x_6, x_7\}$	x_7	x_6	x_2	x_6x_7	x_6	x_7	x_2:r_3, x_6:r_2, x_7:r_1
$\{x_3, x_4, x_6\}$	x_3	x_6	x_4	$x_3x_4x_6$	x_4x_6	x_4	x_3:r_1, x_4:r_3, x_6:r_2
$\{x_4, x_6, x_7\}$	x_7	x_6	x_4	$x_4x_6x_7$	x_4x_6	x_4x_7	x_4:r_3, x_6:r_2, x_7:r_1

Tabla 1: Verificación de los testores típicos de A: todas las intersecciones son no vacías (testor) y cada rasgo tiene fila típica (compatible).

4.4. Ejercicio 2.3: implementación computacional de YYC

Diseño. El algoritmo vive en la clase de dominio `YycSolver`, que expone una interfaz incremental: `processNextRow()` procesa una fila y `current()` devuelve Ψ^*. Esto permite que el caso de uso `runYyc` de la capa de aplicación:

1. lea el reloj monótono antes y después de `processNextRow()` y sume sólo ese intervalo al **tiempo acumulado**;
2. después de cada fila, y *fuera* del intervalo medido, notifique a un *listener* con Ψ^* y el tiempo acumulado. La presentación muestra entonces los testores encontrados hasta ese momento (requisitos 1 y 2 del enunciado) sin que imprimir en pantalla contamine la medición.

La comprobación de compatibilidad recorre las filas $r_1, \ldots, r_k$, calcula $r_i \cap T$ con operaciones de bits y marca los rasgos que aparecen solos; T es compatible si todos quedan marcados. Opcionalmente, el solver registra una traza con cada decisión (qué τ se conservó, qué extensiones se probaron y qué rasgo se quedó sin fila típica); con ella se generó la explicación paso a paso usada para validar la sección anterior.

Interfaz de usuario. El programa `testores` ofrece un menú interactivo (generar, buscar, cargar desde archivo, ingresar a mano, ver la matriz básica y su densidad, ejecutar YYC normal o paso a paso, guardar y exportar resultados) y, para automatizar, subcomandos:

```
testores yyc --entrada data/matriz_A.txt              # Psi* y tiempo tras cada fila
testores yyc --entrada data/matriz_A.txt --traza      # explicación paso a paso
testores yyc --entrada data/matriz_A.txt --repeticiones 10000 \
             --csv resultados/yyc_A.csv --latex resultados/yyc_A_tabla.tex
```

Si la matriz introducida no es básica, el programa lo informa, la reduce (Proposición 2) y aplica YYC a la matriz básica. Un fragmento de la salida (con `Psi*` en lugar del símbolo griego y omitiendo aquí los tiempos, que se presentan en la Tabla 2) es:

```
Tras procesar r3 (3/6): |Psi*| = 7
  Psi* = {x1}, {x2, x5}, {x4, x5}, {x2, x3, x6}, {x2, x6, x7},
         {x3, x4, x6}, {x4, x6, x7}
Tras procesar r4 (4/6): |Psi*| = 12
  ...
```

4.5. Ejercicio 2.4: resultados sobre la matriz A

Una sola ejecución de YYC sobre A dura unos pocos microsegundos, una escala en la que el ruido del sistema operativo (interrupciones, caché fría, cambios de frecuencia) es comparable al propio tiempo medido. Por eso se ejecuta YYC 10 000 veces, se registra en cada ejecución el

tiempo acumulado C_k tras cada fila r_k, y la tabla reporta, para cada fila, la **mediana de** C_k. El último valor es, entonces, la mediana del tiempo total, el mismo estadístico que se usa para BT.

No se suman las medianas de los tiempos de cada fila: la mediana no es aditiva, y esa suma no garantiza representar el tiempo acumulado de una ejecución. La columna sigue siendo creciente: en cada ejecución $C_k \le C_{k+1}$, y la mediana (como cualquier estadístico de orden) conserva esa desigualdad. Los conjuntos de testores son idénticos en todas las ejecuciones y coinciden con la ejecución manual.

Fila	$\lvert\Psi^*\rvert$	Testores típicos de las filas procesadas	T. acum. (μs)
r_1	4	$\{x_1\}$, $\{x_3\}$, $\{x_5\}$, $\{x_7\}$	0.600
r_2	4	$\{x_1\}$, $\{x_5\}$, $\{x_3, x_6\}$, $\{x_6, x_7\}$	1.300
r_3	7	$\{x_1\}$, $\{x_2, x_5\}$, $\{x_4, x_5\}$, $\{x_2, x_3, x_6\}$, $\{x_2, x_6, x_7\}$, $\{x_3, x_4, x_6\}$, $\{x_4, x_6, x_7\}$	2.700
r_4	12	$\{x_1, x_3\}$, $\{x_1, x_4\}$, $\{x_1, x_6\}$, $\{x_1, x_7\}$, $\{x_4, x_5\}$, $\{x_2, x_3, x_5\}$, $\{x_2, x_3, x_6\}$, $\{x_2, x_5, x_6\}$, $\{x_2, x_5, x_7\}$, $\{x_2, x_6, x_7\}$, $\{x_3, x_4, x_6\}$, $\{x_4, x_6, x_7\}$	4.200
r_5	10	$\{x_1, x_3\}$, $\{x_1, x_4\}$, $\{x_1, x_6\}$, $\{x_1, x_7\}$, $\{x_4, x_5\}$, $\{x_2, x_3, x_6\}$, $\{x_2, x_5, x_6\}$, $\{x_2, x_6, x_7\}$, $\{x_3, x_4, x_6\}$, $\{x_4, x_6, x_7\}$	5.500
r_6	8	$\{x_1, x_3\}$, $\{x_1, x_4\}$, $\{x_1, x_6\}$, $\{x_1, x_7\}$, $\{x_4, x_5\}$, $\{x_2, x_6, x_7\}$, $\{x_3, x_4, x_6\}$, $\{x_4, x_6, x_7\}$	7.000

Tabla 2: YYC sobre la matriz A: testores típicos de las filas procesadas y tiempo de ejecución acumulado después de cada fila (mediana de 10 000 ejecuciones).

Interpretación.

- El conjunto final coincide exactamente con el obtenido a mano (8 testores típicos: 5 de cardinalidad 2 y 3 de cardinalidad 3).
- El trabajo por fila depende del número de extensiones y del costo de comprobar su compatibilidad con las filas procesadas. Por ejemplo, r_3 prueba 9 extensiones y r_4 prueba 8, pero la segunda comprueba una fila adicional. La diferencia entre dos medianas acumuladas no debe interpretarse como la mediana del costo individual de esa fila.
- El tiempo sobre A es pequeño, pero esto no implica una cota polinómica. El tamaño de Ψ^* en los pasos intermedios puede crecer exponencialmente con el número de columnas; por eso existe un límite de seguridad sobre $\lvert\Psi^*\rvert$. La densidad por sí sola no determina ese crecimiento.

5. Ejercicio 3: algoritmo BT

5.1. El algoritmo BT

BT fue propuesto por Ruiz-Shulcloper, Bravo y Águila [5]; seguimos la descripción del documento del curso [4]. A diferencia de YYC, que construye los testores típicos a partir de la estructura interna de la matriz, BT es un algoritmo de *escala exterior*: recorre los subconjuntos de rasgos en un orden total fijo y, según el resultado de analizar cada uno, *salta* bloques completos de subconjuntos que no pueden ser testores típicos [6].

Codificación y orden. Cada subconjunto $T \subseteq R$ se representa por su n-uplo característico $\alpha = (a_1, \ldots, a_n) \in \{0,1\}^n$, con $a_j = 1$ si y sólo si $x_j \in T$; escribimos T_α para el conjunto asociado

a α. El n-uplo se lee como un número natural en binario, con a_1 como dígito más significativo:

$$\nu(\alpha) = \sum_{j=1}^{n} a_j \, 2^{\,n-j}.$$

BT analiza los n-uplos en orden creciente de ν, desde $(0,\dots,0,1) = \{x_n\}$ hasta $(1,\dots,1) = R$. Para $n = 3$ el orden es $001 = \{x_3\}$, $010 = \{x_2\}$, $011 = \{x_2, x_3\}$, $100 = \{x_1\}$, 101, 110, 111 (es el mismo que presenta [6, Tabla 4.1]).

Usaremos repetidamente el siguiente hecho elemental: si dos números $\alpha < \alpha'$ coinciden en sus primeros $k-1$ dígitos, todo β con $\alpha < \beta < \alpha'$ también coincide con ellos en esos dígitos (los números con un prefijo fijo forman un intervalo).

Proposición 3 (Salto S1: α no es testor)**.** *Sea α tal que T_α no es testor, sea r_i una fila con $r_i \cap T_\alpha = \emptyset$ y sea $k = \max\{j : r_i[j] = 1\}$ la posición del* último 1 *de r_i. Sea $\alpha' = (a_1,\dots,a_{k-1},1,0,\dots,0)$. Entonces $\alpha < \alpha'$ y ningún β con $\alpha < \beta < \alpha'$ es testor.*

Demostración. Como $r_i[k] = 1$ y r_i no corta a T_α, se tiene $a_k = 0$; luego α y α' coinciden en las posiciones $1,\dots,k-1$ y en la posición k valen 0 y 1, así que $\alpha < \alpha'$. Sea $\alpha < \beta < \alpha'$. Por el hecho anterior, $b_j = a_j$ para $j < k$, y como $\beta < \alpha'$ con el mismo prefijo, $b_k = 0 = a_k$. Las posiciones $j > k$ no importan porque allí $r_i[j] = 0$. Entonces $r_i \cap T_\beta = r_i \cap T_\alpha = \emptyset$: la fila r_i sigue sin cubrirse y T_β no es testor. □

Qué fila usar en S1. La Proposición 3 vale para *cualquier* fila sin cubrir, pero el salto depende de la fila elegida. Si r_i y $r_{i'}$ no están cubiertas y sus últimos unos están en $k < k'$, los destinos α'_k y $\alpha'_{k'}$ coinciden en las posiciones $1,\dots,k-1$ y en la posición k valen 1 y $a_k = 0$, respectivamente; luego $\alpha'_k > \alpha'_{k'}$. Por eso el Paso 3 de [4] indica que, si hay varias filas responsables, *se toma la que tenga el último 1 más a la izquierda*: es la que produce el salto más grande. Si hay empate en esa posición, el salto es el mismo; tomamos la de menor índice.

Proposición 4 (Salto S2: α es testor)**.** *Sea α tal que T_α es testor y sea $k = \max\{j : a_j = 1\}$ la posición del último* 1 *de α. Todo β con $\nu(\alpha) < \nu(\beta) < \nu(\alpha) + 2^{\,n-k}$ cumple $T_\beta \supset T_\alpha$ (inclusión propia); en particular, T_β es testor pero* no *es típico.*

Demostración. Como $a_j = 0$ para $j > k$, $\nu(\alpha)$ es múltiplo de $2^{\,n-k}$ y el intervalo $[\nu(\alpha), \nu(\alpha) + 2^{\,n-k})$ está formado exactamente por los n-uplos que coinciden con α en las posiciones $1,\dots,k$, con cualquier valor en las posiciones $k+1,\dots,n$. Si $\beta \neq \alpha$ está en ese intervalo, tiene algún $b_j = 1$ con $j > k$, luego $T_\beta \supset T_\alpha$ y $T_\beta \neq T_\alpha$. Todo superconjunto de un testor es testor, y no es típico porque contiene propiamente al testor T_α. □

El siguiente candidato tras S2 es $\nu(\alpha) + 2^{\,n-k}$: se "suma 1 en la posición del último 1" con acarreo hacia la izquierda. Si el acarreo se sale por la posición 1 ($\nu(\alpha) + 2^{\,n-k} \geq 2^n$), todos los n-uplos restantes son superconjuntos de T_α y el algoritmo termina. El algoritmo completo es:

1. $\alpha \leftarrow (0,\dots,0,1)$.
2. Buscar todas las filas r_i con $r_i \cap T_\alpha = \emptyset$.
3. **Si existen**, T_α no es testor: elegir entre ellas la fila r_i cuyo último 1 está más a la izquierda; $k \leftarrow$ último 1 de r_i; $\alpha \leftarrow (a_1,\dots,a_{k-1},1,0,\dots,0)$ y volver a 2.
4. **Si no existen**, T_α es testor: registrarlo y decidir si es típico; $k \leftarrow$ último 1 de α; $\alpha \leftarrow \alpha + 2^{\,n-k}$. Si hubo desbordamiento, terminar; si no, volver a 2.

Si la matriz tiene una fila nula, esa fila no está cubierta por ningún α y no tiene último 1: ningún n-uplo es testor, así que BT termina en el primer paso sin testores (la implementación la trata como la fila "más a la izquierda" posible).

Proposición 5 (Corrección de BT)**.** *BT termina y analiza todos los testores típicos de la matriz.*

Demostración. En cada paso $\nu(\alpha)$ crece estrictamente y está acotado por $2^n - 1$, así que BT termina tras a lo sumo $2^n - 1$ pasos. Los únicos n-uplos que no se analizan son los que se saltan en S1 (no son testores, Proposición 3), los que se saltan en S2 y los que quedan tras el desbordamiento final (superconjuntos propios de un testor, Proposición 4). Ninguno de ellos es testor típico. □

Identificación de los testores típicos. BT, por sí solo, encuentra testores, y entre ellos hay testores no típicos (los superconjuntos de un testor que no son "consecutivos" a él en el orden). El enunciado pide un procedimiento para identificar los típicos; implementamos dos, que se comprueban entre sí:

1. **Durante la ejecución.** Cuando T_α resulta testor, se comprueba si cada rasgo de T_α tiene una fila típica; por la Proposición 1, esto equivale a que T_α sea típico.
2. **A posteriori.** Al terminar, se toman los elementos minimales (respecto de $\subseteq$) de la familia $\mathcal{F}$ de testores encontrados.

Proposición 6. *Los testores típicos de la matriz son exactamente los elementos minimales de $\mathcal{F}$.*

Demostración. Si T es típico, $T \in \mathcal{F}$ (Proposición 5) y ningún subconjunto propio $T' \subset T$ es testor, así que T es minimal en $\mathcal{F}$. Recíprocamente, sea T minimal en $\mathcal{F}$. Quitando rasgos a T mientras siga siendo testor se llega a un testor típico $T_0 \subseteq T$, que también está en $\mathcal{F}$; por minimalidad, $T_0 = T$. □

Orden de las filas. Como la fila del salto S1 se elige por la posición de su último 1 y no por su posición en la matriz, la secuencia de n-uplos que analiza BT *no depende del orden de las filas*: reordenarlas sólo puede cambiar cuál de dos filas empatadas se cita, pero no el salto. El precio es que, para elegir la fila, hay que revisar todas las filas sin cubrir y no sólo la primera. Una implementación que tomara siempre la primera fila sin cubrir sí dependería del orden de las filas y daría saltos más cortos (en la matriz A, 51 n-uplos analizados en lugar de 44; ver la Sección 5.2). Nuestra implementación no reordena filas ni columnas; el efecto del orden de las filas sobre los tiempos se estudia en el Ejercicio 5.

5.2. Ejercicio 3.1: ejecución manual de BT sobre A

En la notación de n-uplos, las filas de A y la posición de su último 1 son

	r_1	r_2	r_3	r_4	r_5	r_6
fila	1010101	1000110	1101000	0011011	1001010	1001001
último 1	x_7	x_6	x_4	x_7	x_6	x_7

Para cada candidato α se buscan todas las filas con ceros en todas las columnas de T_α; si hay varias, la tabla anterior indica cuál tiene el último 1 más a la izquierda. Detallamos los primeros pasos y los más ilustrativos; la Tabla 3 contiene el recorrido completo, con la justificación de cada decisión.

Paso 1. $\alpha = 0000001$, $T_\alpha = \{x_7\}$. No contienen a x_7 las filas $r_2 = \{x_1, x_5, x_6\}$ (último 1 en x_6), $r_3 = \{x_1, x_2, x_4\}$ (en x_4) y $r_5 = \{x_1, x_4, x_6\}$ (en x_6): no es testor. La de último 1 más a la izquierda es r_3, así que (S1) $\alpha' = 0001000$. Se saltan los 6 n-uplos $0000010, \ldots, 0000111$: todos tienen ceros en x_1, x_2, x_4 y no cubren r_3. (Con la primera fila sin cubrir, r_2, el salto habría llegado sólo a 0000010.)

Paso 2. $\{x_4\}$: quedan sin cubrir r_1 (último 1 en x_7) y r_2 (en x_6); se elige r_2 y $\alpha' = 0001010$. Se salta $0001001 = \{x_4, x_7\}$, que tampoco cubre r_2.

Paso 3. $\{x_4, x_6\}$: sólo r_1 queda sin cubrir; su último 1 está en x_7, así que $\alpha' = 0001011$.

Paso 4. $\{x_4, x_6, x_7\}$: $r_1 \to x_7$, $r_2 \to x_6$, $r_3 \to x_4$, $r_4 \to \{x_4, x_6, x_7\}$, $r_5 \to \{x_4, x_6\}$, $r_6 \to \{x_4, x_7\}$. Ninguna intersección es vacía: **es testor**. Además r_3, r_2 y r_1 son filas típicas de x_4, x_6 y x_7: **es típico**. El último 1 está en x_7 (S2): $\alpha' = 0001011 + 1 = 0001100$.

Paso 5. $\{x_4, x_5\}$: $r_1, r_2 \to x_5$ y $r_3, \ldots, r_6 \to x_4$; es testor y típico (x_4:r_3, x_5:r_1). El último 1 está en x_5: $\alpha' = 0001100 + 0000100 = 0010000$. Se saltan 0001101, 0001110 y 0001111, que son $\{x_4, x_5\}$ más x_6 y/o x_7.

Paso 6. $\{x_3\}$: quedan sin cubrir r_2, r_3, r_5 y r_6; otra vez r_3 tiene el último 1 más a la izquierda (x_4) y $\alpha' = 0011000$: se saltan 7 n-uplos.

Paso 9. $\{x_3, x_4, x_5\}$ es testor pero **no típico**: x_3 sólo aparece en r_1 (junto a x_5) y en r_4 (junto a x_4), así que no tiene fila típica. En efecto, contiene al testor $\{x_4, x_5\}$ encontrado en el paso 5, pero no es consecutivo a él en el orden y por eso S2 no lo evitó.

Pasos 35–36. $\{x_1, x_4\}$ y $\{x_1, x_3\}$ son típicos y producen los saltos más grandes: 1001000 + 0001000 = 1010000 (7 superconjuntos omitidos) y 1010000+0010000 = 1100000 (15 omitidos).

Paso 44. $\{x_1, x_2, x_3\}$ es testor no típico (x_2 sólo aparece en r_3, que también contiene a x_1). Al sumar 1 en la posición de x_3 el acarreo se sale por la izquierda (1110000+0010000 = 10000000): los 15 n-uplos restantes son superconjuntos de $\{x_1, x_2, x_3\}$ y **el algoritmo termina**.

Resumen de la ejecución.

- BT analizó **44** de los $2^7 - 1 = 127$ subconjuntos no vacíos y saltó los otros 83: 20 candidatos no fueron testores (salto S1) y 24 sí lo fueron (salto S2).
- La elección de la fila en S1 importa: si se tomara siempre la primera fila sin cubrir, BT analizaría 51 n-uplos (los mismos 24 testores y 27 no testores). La regla del último 1 más a la izquierda ahorra 7 análisis, todos de subconjuntos que no son testores. Los testores analizados son los mismos con cualquier regla, porque S1 sólo salta subconjuntos que no son testores y S2 no depende de las filas.
- De los 24 testores visitados, **8 son típicos**: $\{x_4, x_6, x_7\}$, $\{x_4, x_5\}$, $\{x_3, x_4, x_6\}$, $\{x_2, x_6, x_7\}$, $\{x_1, x_7\}$, $\{x_1, x_6\}$, $\{x_1, x_4\}$, $\{x_1, x_3\}$ (en el orden en que BT los encuentra). Es exactamente el conjunto obtenido con YYC en la Sección 4.3. Los 16 testores restantes no son típicos, y los elementos minimales de la familia de 24 testores son, de nuevo, esos 8 (Proposición 6). En total A tiene **86 testores**, según la enumeración exhaustiva de los 127 subconjuntos no vacíos: BT no pretende visitarlos todos, porque S2 omite superconjuntos no típicos.
- La debilidad de BT señalada en [6] se ve claramente: dos tercios de los testores analizados son superconjuntos de testores típicos (por ejemplo $\{x_2, x_4, x_5\} \supset \{x_4, x_5\}$) que el salto S2 no puede evitar, porque no son consecutivos al testor en el orden binario. Como comparación, YYC probó 35 extensiones y sólo conservó los conjuntos compatibles con las filas procesadas.

5.3. Ejercicio 3.2: implementación computacional de BT

Diseño. BT vive en la capa de dominio (`domain/bt.hpp`). Los saltos son dos funciones puras: `btJumpNotTestor` (S1: poner 1 en la posición k y ceros a su derecha) y `btJumpTestor` (S2: sumar 1 en la posición del último 1 con acarreo; devuelve "vacío" si hay desbordamiento). `BtSolver::solve` repite: recorrer las filas con operaciones `AND` sobre palabras de 64 bits y quedarse con la fila sin cubrir cuyo último 1 está más a la izquierda (función `btZeroRow`); si no hay ninguna, registrar el testor, decidir su tipicidad (procedimiento 1) y aplicar S2; si la hay, aplicar S1 (o terminar sin testores si esa fila es nula). El resultado contiene la lista de testores encontrados (en el orden de BT y marcados como típicos o no), los testores típicos y, opcionalmente, la traza de cada paso. El caso de uso `runBt` lee el reloj monótono antes y después de `solve`, de modo que el **tiempo total** medido no incluye la impresión; fuera de la medición aplica el procedimiento 2 (minimales) y comprueba que coincide con el 1. Como con YYC, `benchmarkBt` repite la ejecución y reporta la mediana. Hay cotas de seguridad sobre la cantidad de testores y de pasos de traza guardados.

Paso	α	T_α	Decisión	Justificación	Salto	α'
1	0000001	$\{x_7\}$	no testor	sin cubrir: $r_2, \mathbf{r_3}, r_5$	S1: x_4	0001000
2	0001000	$\{x_4\}$	no testor	sin cubrir: $r_1, \mathbf{r_2}$	S1: x_6	0001010
3	0001010	$\{x_4, x_6\}$	no testor	sin cubrir: $\mathbf{r_1}$	S1: x_7	0001011
4	0001011	$\{x_4, x_6, x_7\}$	**típico**	x_4:r_3, x_6:r_2, x_7:r_1	S2: x_7	0001100
5	0001100	$\{x_4, x_5\}$	**típico**	x_4:r_3, x_5:r_1	S2: x_5	0010000
6	0010000	$\{x_3\}$	no testor	sin cubrir: $r_2, \mathbf{r_3}, r_5, r_6$	S1: x_4	0011000
7	0011000	$\{x_3, x_4\}$	no testor	sin cubrir: $\mathbf{r_2}$	S1: x_6	0011010
8	0011010	$\{x_3, x_4, x_6\}$	**típico**	x_3:r_1, x_4:r_3, x_6:r_2	S2: x_6	0011100
9	0011100	$\{x_3, x_4, x_5\}$	no típico	x_3 sin fila típica	S2: x_5	0100000
10	0100000	$\{x_2\}$	no testor	sin cubrir: $r_1, \mathbf{r_2}, r_4, r_5, r_6$	S1: x_6	0100010
11	0100010	$\{x_2, x_6\}$	no testor	sin cubrir: $\mathbf{r_1}, r_6$	S1: x_7	0100011
12	0100011	$\{x_2, x_6, x_7\}$	**típico**	x_2:r_3, x_6:r_2, x_7:r_1	S2: x_7	0100100
13	0100100	$\{x_2, x_5\}$	no testor	sin cubrir: $r_4, \mathbf{r_5}, r_6$	S1: x_6	0100110
14	0100110	$\{x_2, x_5, x_6\}$	no testor	sin cubrir: $\mathbf{r_6}$	S1: x_7	0100111
15	0100111	$\{x_2, x_5, x_6, x_7\}$	no típico	x_5 sin fila típica	S2: x_7	0101000
16	0101000	$\{x_2, x_4\}$	no testor	sin cubrir: $r_1, \mathbf{r_2}$	S1: x_6	0101010
17	0101010	$\{x_2, x_4, x_6\}$	no testor	sin cubrir: $\mathbf{r_1}$	S1: x_7	0101011
18	0101011	$\{x_2, x_4, x_6, x_7\}$	no típico	x_2 sin fila típica	S2: x_7	0101100
19	0101100	$\{x_2, x_4, x_5\}$	no típico	x_2 sin fila típica	S2: x_5	0110000
20	0110000	$\{x_2, x_3\}$	no testor	sin cubrir: $\mathbf{r_2}, r_5, r_6$	S1: x_6	0110010
21	0110010	$\{x_2, x_3, x_6\}$	no testor	sin cubrir: $\mathbf{r_6}$	S1: x_7	0110011
22	0110011	$\{x_2, x_3, x_6, x_7\}$	no típico	x_3 sin fila típica	S2: x_7	0110100
23	0110100	$\{x_2, x_3, x_5\}$	no testor	sin cubrir: $\mathbf{r_5}, r_6$	S1: x_6	0110110
24	0110110	$\{x_2, x_3, x_5, x_6\}$	no testor	sin cubrir: $\mathbf{r_6}$	S1: x_7	0110111
25	0110111	$\{x_2, x_3, x_5, x_6, x_7\}$	no típico	x_3 sin fila típica	S2: x_7	0111000
26	0111000	$\{x_2, x_3, x_4\}$	no testor	sin cubrir: $\mathbf{r_2}$	S1: x_6	0111010
27	0111010	$\{x_2, x_3, x_4, x_6\}$	no típico	x_2 sin fila típica	S2: x_6	0111100
28	0111100	$\{x_2, x_3, x_4, x_5\}$	no típico	x_2 sin fila típica	S2: x_5	1000000
29	1000000	$\{x_1\}$	no testor	sin cubrir: $\mathbf{r_4}$	S1: x_7	1000001
30	1000001	$\{x_1, x_7\}$	**típico**	x_1:r_2, x_7:r_4	S2: x_7	1000010
31	1000010	$\{x_1, x_6\}$	**típico**	x_1:r_1, x_6:r_4	S2: x_6	1000100
32	1000100	$\{x_1, x_5\}$	no testor	sin cubrir: $\mathbf{r_4}$	S1: x_7	1000101
33	1000101	$\{x_1, x_5, x_7\}$	no típico	x_5 sin fila típica	S2: x_7	1000110
34	1000110	$\{x_1, x_5, x_6\}$	no típico	x_5 sin fila típica	S2: x_6	1001000
35	1001000	$\{x_1, x_4\}$	**típico**	x_1:r_1, x_4:r_4	S2: x_4	1010000
36	1010000	$\{x_1, x_3\}$	**típico**	x_1:r_2, x_3:r_4	S2: x_3	1100000
37	1100000	$\{x_1, x_2\}$	no testor	sin cubrir: $\mathbf{r_4}$	S1: x_7	1100001
38	1100001	$\{x_1, x_2, x_7\}$	no típico	x_2 sin fila típica	S2: x_7	1100010
39	1100010	$\{x_1, x_2, x_6\}$	no típico	x_2 sin fila típica	S2: x_6	1100100
40	1100100	$\{x_1, x_2, x_5\}$	no testor	sin cubrir: $\mathbf{r_4}$	S1: x_7	1100101
41	1100101	$\{x_1, x_2, x_5, x_7\}$	no típico	x_2 sin fila típica	S2: x_7	1100110
42	1100110	$\{x_1, x_2, x_5, x_6\}$	no típico	x_2 sin fila típica	S2: x_6	1101000
43	1101000	$\{x_1, x_2, x_4\}$	no típico	x_2 sin fila típica	S2: x_4	1110000
44	1110000	$\{x_1, x_2, x_3\}$	no típico	x_2 sin fila típica	S2: x_3	fin

Tabla 3: Recorrido completo de BT sobre A. En los no testores se listan las filas sin cubrir y en negrita la elegida (la de último 1 más a la izquierda). "S1: x_k" indica el salto por fila de ceros (1 en x_k y ceros a su derecha); "S2: x_k", el salto por testor (sumar 1 en la posición x_k). En los testores típicos se indica una fila típica de cada rasgo.

Interfaz de usuario. En el menú interactivo se añadieron las opciones 10 (ejecutar BT), 11 (BT paso a paso con explicación) y 12 (exportar el último resultado), que trabajan sobre la matriz actual (generada con el Ejercicio 1, cargada de archivo o ingresada a mano). Si la matriz no es básica, se reduce primero. También hay un subcomando:

```
testores bt --entrada data/matriz_A.txt          # testores, típicos, tiempo
testores bt --entrada data/matriz_A.txt --traza  # cada alfa y cada salto
testores bt --entrada data/matriz_A.txt --repeticiones 10000 \
            --csv resultados/bt_A.csv --latex resultados/bt_A_resumen.tex
```

Un fragmento de la salida:

```
Paso 1: alfa = 0000001 = {x7}
  NO es testor: filas con ceros en todas las columnas de alfa:
  r2 (ultimo 1 en x6), r3 (ultimo 1 en x4), r5 (ultimo 1 en x6).
  Se elige r3 = 1101000, la de ultimo 1 mas a la izquierda (salto mas grande).
  Salto S1: ultimo 1 de r3 en x4 -> poner 1 en x4 y ceros a su derecha.
  Siguiente: alfa' = 0001000 = {x4}  (se saltan 6 n-uplos)
...
Paso 9: alfa = 0011100 = {x3, x4, x5}
  ES testor (toda fila tiene un 1 en alguna columna de alfa) y NO es tipico:
  x3 no tiene fila tipica (se puede quitar y sigue siendo testor).
  Salto S2: ultimo 1 de alfa en x5 -> sumar 1 en esa posicion
  Siguiente: alfa' = 0100000 = {x2}  (se saltan 3 n-uplos)
...
Resultados de BT:
  n-uplos analizados: 44 de 127 posibles (2^7 - 1); se saltaron 83
  Testores encontrados: 24 (tipicos: 8, no tipicos: 16)
  Ambos procedimientos coinciden
```

Verificación de BT. Las pruebas específicas de BT incluyen los siguientes casos:

- la traza exacta (n-uplos visitados, filas de ceros y saltos) sobre una matriz de 3×3 calculada a mano, y los 44 pasos sobre A (en el primero se elige r_3, no r_2);
- en cada paso no testor de 150 matrices aleatorias, la fila elegida no está cubierta y tiene el último 1 más a la izquierda entre las no cubiertas;
- reordenar las filas (en A y en 100 matrices aleatorias) no cambia la secuencia de n-uplos analizados;
- con una fila nula, BT, YYC y la matriz básica dan cero testores;
- BT coincide con el oráculo exhaustivo y con YYC en 500 matrices básicas aleatorias de 1 a 12 columnas;
- en 150 matrices aleatorias se recorren *todos* los n-uplos que BT no analizó y se comprueba que ninguno es testor típico, que el orden de visita es estrictamente creciente y que cada testor se clasificó correctamente; es una comprobación empírica de las Proposiciones 3 y 4;
- los minimales de los testores encontrados coinciden con los típicos (Proposición 6).

Además, la traza de la Tabla 3 se reprodujo con una implementación independiente de BT escrita en Python a partir de las Proposiciones 1.3–1.5 y el Paso 3 de [4]: los 44 pasos coinciden (n-uplos, filas elegidas y saltos).

5.4. Ejercicio 3.3: resultados sobre la matriz A

Interpretación.

- Los testores típicos coinciden exactamente con los de la ejecución manual y con los de YYC (Tabla 2).

n-uplos analizados	44 de $2^7 - 1 = 127$
Testores encontrados	24 (8 típicos, 16 no típicos)
Testores típicos	$\{x_1, x_3\}$, $\{x_1, x_4\}$, $\{x_1, x_6\}$, $\{x_1, x_7\}$, $\{x_4, x_5\}$, $\{x_2, x_6, x_7\}$, $\{x_3, x_4, x_6\}$, $\{x_4, x_6, x_7\}$
Tiempo total (μs)	10.000

Tabla 4: BT sobre la matriz A: testores típicos obtenidos y tiempo total de ejecución (mediana de 10 000 ejecuciones). La lista completa de los 24 testores encontrados está en `docs/resultados/bt_A.csv`.

- Para A, la mediana acumulada final de YYC es 7.000 μs y la mediana total de BT es 10.000 μs, según los CSV regenerados. Esas cifras describen sus respectivos intervalos de medición, que se detallan a continuación.
- Las tablas de los Ejercicios 2 y 3 muestran el tiempo acumulado por fila de YYC y el tiempo total de BT, respectivamente. No se atribuye una diferencia de tiempos solamente a los conteos 35 y 44: una extensión de YYC y un candidato de BT realizan operaciones distintas.
- La medición por fila de YYC excluye la construcción del solver; la de BT engloba su resolución completa. La comparación con el mismo alcance de medición, para ambos órdenes de filas, se presenta en el Ejercicio 5.

6. Ejercicio 4: construcción y análisis de la matriz B

6.1. Ejercicio 4.1: construcción de la matriz B

Los algoritmos YYC y BT obtuvieron los mismos ocho testores típicos de la matriz A, de acuerdo con los resultados presentados en los ejercicios anteriores. Utilizamos la notación $\Psi^*(M)$ para representar el conjunto de testores típicos de una matriz M, siguiendo la terminología de [1, 9]. En este caso,

$$\Psi^*(A) = \{\{x_1, x_3\}, \{x_1, x_4\}, \{x_1, x_6\}, \{x_1, x_7\}, \\ \{x_4, x_5\}, \{x_2, x_6, x_7\}, \{x_3, x_4, x_6\}, \{x_4, x_6, x_7\}\}.$$

Para construir B, convertimos cada testor típico en una fila binaria de siete posiciones, correspondientes a las características $x_1, \ldots, x_7$, como pide el ejercicio 4.1 del proyecto [10]. Escribimos un 1 cuando la característica pertenece al testor y un 0 cuando no pertenece. Por ejemplo,

$$\{x_1, x_3\} \longrightarrow (1, 0, 1, 0, 0, 0, 0).$$

Al convertir los ocho testores típicos, obtenemos la matriz

$$B = \begin{pmatrix} 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 \end{pmatrix}.$$

La matriz B tiene ocho filas, siete columnas y diecinueve unos. Por tanto, su densidad es

$$\rho(B) = \frac{19}{8 \cdot 7} = \frac{19}{56} \approx 0.3393.$$

Este resultado cumple el enunciado, ya que la condición de densidad mayor o igual a 0.4 se exigió para A, no para B [10].

Demostración de que B es básica

Para demostrar que B es una matriz básica, verificamos que no tiene filas nulas, filas repetidas ni superfilas. La ausencia de repeticiones y superfilas demuestra la basicidad; adicionalmente, se comprueba que no existen filas nulas, por lo que la matriz admite testores. Se utiliza la definición de matriz básica de la teoría de testores [5, 9].

1. **Ausencia de filas nulas.** Una fila nula es aquella cuyas entradas son todas iguales a cero. En B, las primeras cinco filas contienen dos unos y las tres restantes contienen tres unos. Por lo tanto, ninguna de sus filas es nula.

2. **Ausencia de filas repetidas.** Los ocho testores típicos de A utilizados para construir B son distintos. Como cada conjunto tiene una representación binaria única para el orden de columnas $x_1, x_2, \ldots, x_7$, las filas correspondientes también son distintas. Por tanto, B no contiene filas repetidas.

3. **Ausencia de superfilas.** Una fila es superfila de otra si contiene unos en todas las posiciones donde la otra tiene unos y, además, tiene al menos un uno adicional. En términos de conjuntos, esto significa que el conjunto representado por una fila contiene estrictamente al representado por la otra.

   Supongamos que en B existe una fila que es superfila de otra. Si los testores típicos de A representados por estas filas son T_j y T_i, respectivamente, entonces

   $$T_i \subsetneq T_j.$$

   Como T_i es un testor de A, sus características son suficientes para cubrir todas las filas de A. Por tanto, T_j contiene un subconjunto propio que ya es testor. Esto contradice la tipicidad de T_j, pues un testor típico es minimal por inclusión, es decir, ninguno de sus subconjuntos propios puede ser testor. En consecuencia, ninguna fila de B es superfila de otra.

Por lo tanto, B no tiene filas repetidas ni superfilas y se concluye que B **es una matriz básica**. Este resultado también concuerda con el lema 2.18 presentado por Guevara Romero [9], según el cual la matriz construida con los testores típicos de una matriz básica es igualmente básica.

6.2. Ejercicio 4.2: aplicación del algoritmo BT sobre B

Se utilizó la implementación del algoritmo BT para procesar la matriz B, almacenada, según el texto de trabajo, en el archivo `data/matriz_B.txt`. Se indica el siguiente comando de ejecución:

```
testores bt --entrada data/matriz_B.txt --completo
```

En la ejecución reportada, BT analizó 44 de los $2^7 - 1 = 127$ subconjuntos no vacíos posibles. Mediante sus reglas de salto, omitió los 83 candidatos restantes, siguiendo la estrategia de búsqueda del algoritmo BT [5]. Durante ese recorrido se reportaron 20 testores, de los cuales 6 son típicos y 14 no típicos. Los seis testores típicos de B, comprobados también a partir de la matriz binaria, se presentan en la Tabla 5.

La tipicidad se verificó mediante dos procedimientos. Durante la ejecución se comprobó que cada característica del testor tuviera una fila típica, es decir, una fila cubierta únicamente por esa característica dentro del testor. Posteriormente, se identificaron los conjuntos minimales por inclusión entre los testores encontrados. Ambos procedimientos produjeron los mismos seis testores típicos. El criterio de las filas típicas coincide con la caracterización teórica de tipicidad [1, 9].

Número	Testor típico de B	Representación binaria
1	$\{x_1, x_2, x_4\}$	`1101000`
2	$\{x_1, x_4, x_6\}$	`1001010`
3	$\{x_1, x_4, x_7\}$	`1001001`
4	$\{x_1, x_5, x_6\}$	`1000110`
5	$\{x_1, x_3, x_5, x_7\}$	`1010101`
6	$\{x_3, x_4, x_6, x_7\}$	`0011011`

Tabla 5: Testores típicos de la matriz B.

El tiempo de 41.8 μs corresponde al registro manual de la versión previa de BT. La versión corregida del programa utiliza la regla del último 1 más a la izquierda y su salida reproducible se conserva en `docs/resultados/bt_B.txt`; las mediciones de distintas versiones no se mezclan.

6.3. Ejercicio 4.3: comparación entre A y B

Comparación de las matrices

Las matrices A y B son básicas y tienen siete columnas, que representan las mismas características $x_1, \ldots, x_7$. Sin embargo, difieren en su número de filas, densidad y cantidad de testores típicos. La Tabla 6 resume estas propiedades.

Propiedad	Matriz A	Matriz B
Número de filas	6	8
Número de columnas	7	7
Cantidad de unos	20	19
Densidad	$20/42 \approx 0.4762$	$19/56 \approx 0.3393$
Número de testores típicos	8	6
Tamaños de los testores típicos	2 y 3	3 y 4
Es matriz básica	Sí	Sí

Tabla 6: Comparación de las matrices A y B.

La matriz B tiene ocho filas porque se construyó a partir de los ocho testores típicos de A. Cada fila indica, mediante sus unos, las características que pertenecen al testor correspondiente. Aunque ambas matrices son básicas, sus densidades son diferentes. Por tanto, en este caso la construcción conserva el número de columnas y la basicidad, pero no conserva el número de filas ni la densidad.

Relación entre las filas y los testores típicos

Al calcular los testores típicos de B, se obtuvieron seis conjuntos. Sus representaciones binarias coinciden exactamente con las seis filas de A, aunque aparecen en un orden diferente, como se observa en la Tabla 7.

Para expresar esta relación, denotamos por $\mathcal{R}(M)$ la familia de conjuntos representados por las filas de una matriz M, y por $\Psi^*(M)$ su conjunto de testores típicos. Por la construcción de B y los resultados obtenidos, se cumple

$$\mathcal{R}(B) = \Psi^*(A), \qquad \mathcal{R}(A) = \Psi^*(B).$$

Esta relación es una dualidad entre las restricciones representadas por las filas y las selecciones minimales de características que permiten satisfacerlas. También aparece como propiedad de doble aplicación de los testores típicos en el lema 2.19 de [9].

Fila de A	Representación binaria	Testor típico de B
r_1	1010101	$\{x_1, x_3, x_5, x_7\}$
r_2	1000110	$\{x_1, x_5, x_6\}$
r_3	1101000	$\{x_1, x_2, x_4\}$
r_4	0011011	$\{x_3, x_4, x_6, x_7\}$
r_5	1001010	$\{x_1, x_4, x_6\}$
r_6	1001001	$\{x_1, x_4, x_7\}$

Tabla 7: Correspondencia entre las filas de A y los testores típicos de B.

Justificación de la dualidad

Sea $X = \{x_1, \ldots, x_7\}$ el conjunto de características. Interpretamos cada fila como el conjunto de columnas donde tiene unos.

Primero, cualquier fila R de A intersecta todos los testores típicos de A, porque estos deben cubrir todas las filas. Como dichos testores constituyen las filas de B, el conjunto R es un testor de B. Cualquier conjunto que contenga una fila de A también será testor de B.

Ahora demostraremos que todo testor de B debe contener alguna fila completa de A. Supongamos que un conjunto S no contiene ninguna fila completa de A. Entonces cada fila de A tiene al menos una característica fuera de S. Por tanto, el complemento $X \setminus S$ cubre todas las filas de A y es un testor de esta matriz.

Al eliminar las características que sobren de ese complemento, se obtiene un testor típico T de A tal que

$$T \subseteq X \setminus S.$$

El conjunto T es una fila de B y no tiene características en común con S. Por ello, S no cubre esa fila y no puede ser testor de B.

Así, los testores de B son precisamente los conjuntos que contienen alguna fila de A. Como A es básica, ninguna de sus filas contiene a otra. En consecuencia, sus filas son exactamente los testores minimales por inclusión de B, es decir, sus testores típicos.

Conclusiones

Los conjuntos de testores típicos de A y B son diferentes, tanto en su cantidad como en sus elementos. Sin embargo, existe una correspondencia recíproca entre las filas de una matriz y los testores típicos de la otra.

Si se construye una nueva matriz C utilizando las representaciones binarias de todos los testores típicos de B, se recupera A, salvo una posible permutación de sus filas. Por tanto, esta construcción es reversible para la matriz básica estudiada. La condición de basicidad de A garantiza que todas sus filas son necesarias en esta correspondencia, pues ninguna es redundante por repetición o por ser superfila de otra.

La construcción conserva las columnas y la basicidad, pero en este caso cambia el número de filas, la densidad y la cantidad de testores típicos. Aunque B tiene más filas que A, posee menos testores típicos. Por ello, tener más filas no implica necesariamente tener más testores típicos.

7. Ejercicio 5

7.1. Ejercicio 5.1: implementación de los operadores y generación de matrices

En este ejercicio se implementaron los operadores φ, θ y γ descritos por Alba-Cabrera et al. [2]. Su propósito es construir matrices básicas con dimensiones controladas y con un número de testores típicos que puede calcularse de antemano. Esto permite preparar experimentos cuya respuesta esperada es conocida, en lugar de elegir matrices aleatorias sin una referencia para comprobar los resultados.

Se conserva el procedimiento de las Tablas 5 y 6 del artículo, pero se utilizan las matrices A y B de nuestros ejercicios 2.1 y 4.1. Por tanto, se reproducen las familias de operaciones y los valores de N, no los valores numéricos de las matrices originales del artículo. La generación de matrices corresponde al punto 5.1; sus tiempos de resolución con YYC y BT corresponden al punto 5.2.

Definición de los operadores

Sean U una matriz básica de $m \times n$ y V una matriz básica de $p \times q$. Al unirlas, las columnas de V ocupan posiciones nuevas, desde $n+1$ hasta $n+q$. No se identifican con las columnas de U, aunque en sus archivos originales ambas matrices comiencen por x_1.

Operador θ: todas las parejas de filas. Cada fila de U se concatena horizontalmente con cada fila de V:

$$\theta(U,V) = \begin{pmatrix} u_1 \mid v_1 \\ u_1 \mid v_2 \\ \vdots \\ u_1 \mid v_p \\ u_2 \mid v_1 \\ \vdots \\ u_m \mid v_p \end{pmatrix}.$$

La notación $u_i \mid v_j$ significa escribir primero los n valores de u_i y después los q valores de v_j; la barra sólo separa visualmente los bloques. El resultado tiene mp filas y $n+q$ columnas. No requiere que las entradas tengan el mismo número de filas.

El procedimiento computacional recorre las filas de U con un ciclo externo y las de V con un ciclo interno. Así produce, en ese orden, $(u_1 \mid v_1), \ldots, (u_1 \mid v_p), (u_2 \mid v_1), \ldots$. No se utiliza la multiplicación matricial usual ni el producto de Kronecker: la operación definida aquí es una concatenación para cada pareja de filas.

Operador φ: concatenación fila por fila. Requiere $m = p$. Une la primera fila de U con la primera de V, la segunda con la segunda, y así sucesivamente:

$$\varphi(U,V) = [\, U \mid V \,] = \begin{pmatrix} u_1 \mid v_1 \\ u_2 \mid v_2 \\ \vdots \\ u_m \mid v_m \end{pmatrix}.$$

El resultado tiene m filas y $n+q$ columnas. A diferencia de θ, no genera todas las parejas posibles. Para obtener $\varphi^N(U)$ se colocan N copias de U horizontalmente:

$$\varphi^N(U) = [\underbrace{U \mid U \mid \cdots \mid U}_{N \text{ copias}}].$$

Así se conservan las m filas y se obtienen Nn columnas.

Operador γ: bloques diagonales. Coloca U arriba a la izquierda y V abajo a la derecha, y completa con ceros los otros dos bloques:

$$\gamma(U,V) = \begin{pmatrix} U & 0_{m\times q} \\ 0_{p\times n} & V \end{pmatrix}.$$

Su tamaño es $(m+p)\times(n+q)$. Para $\gamma^N(U)$ se colocan N copias de U en la diagonal y ceros fuera de esos bloques. En este caso se obtienen Nm filas y Nn columnas.

En la notación del artículo, N cuenta las copias de la matriz, no el número de llamadas adicionales a la función. Por ello, $\varphi^1(U) = \gamma^1(U) = U$ y $\gamma^2(U) = \gamma(U,U)$.

Ejemplo para distinguir las tres construcciones

Considérese la matriz pequeña $P = I_2$, independiente de las matrices A y B del proyecto:

$$P = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}.$$

Aplicando cada operador a dos copias de P, se obtiene:

$$\theta(P,P) = \begin{pmatrix} 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{pmatrix}, \qquad \varphi(P,P) = \begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{pmatrix}, \qquad \gamma(P,P) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

Las tres matrices tienen cuatro columnas, pero se construyen de manera distinta: θ combina todas las parejas, φ une filas correspondientes y γ separa las copias en bloques diagonales. En este ejemplo tienen, respectivamente, 4, 2 y 4 filas.

Conservación de la basicidad

Los operadores conservan la condición de matriz básica cuando sus entradas son básicas y no contienen filas nulas [2]. Esta propiedad puede entenderse comparando los bloques de las filas:

- En $\theta(U,V)$, si una fila fuera subfila de otra, su bloque de U tendría que estar contenido en el otro bloque de U, y lo mismo ocurriría con los bloques de V. La basicidad obliga a que ambos bloques sean iguales, por lo que no existe una inclusión estricta. Además, parejas distintas no producen filas repetidas.
- En $\varphi(U,V)$, dos filas diferentes ya son incomparables en el bloque correspondiente a U. Agregar el bloque de V no elimina esa diferencia ni puede convertirlas en filas repetidas.
- En $\gamma(U,V)$, dentro de cada bloque se conserva la incomparabilidad original. Entre bloques distintos, cada fila tiene algún 1 en posiciones donde la otra tiene sólo ceros.

Las filas tampoco son nulas, porque cada una conserva una fila no nula de alguna matriz de entrada. Por tanto, no es necesario reducir ni reordenar los resultados antes de usarlos en los experimentos. Esta afirmación se comprobó también con la rutina de matriz básica.

Implementación computacional

La implementación está escrita en C++17 y utiliza la representación `BooleanMatrix`. Cada fila es un `ColumnSet`, que almacena las posiciones de sus unos. Las funciones públicas se declaran en `include/testores/domain/matrix_operators.hpp` y se implementan en `src/domain/matrix_operators.cpp`.

Función	Responsabilidad
`theta(U,V)`	Generar todas las concatenaciones $u_i \mid v_j$.
`phi(U,V)`	Concatenar las filas correspondientes, comprobando que haya igual cantidad de filas.
`phiPower(U,N)`	Construir directamente las N copias horizontales de U.
`gamma(U,V)`	Construir los dos bloques diagonales y sus ceros.
`gammaPower(U,N)`	Construir directamente N bloques diagonales de U.

Para concatenar un bloque a la derecha, las posiciones de sus unos se desplazan por el ancho del bloque izquierdo. Por ejemplo, si éste tiene siete columnas, la primera columna del bloque derecho ocupa la posición x_8. En las potencias de γ, las posiciones fuera del bloque activo permanecen en cero.

Las funciones rechazan entradas vacías, cero copias y resultados que excedan los límites de 100 000 filas o 512 columnas. La validación de dimensiones se realiza antes de reservar la memoria. La interfaz de comandos exige entradas básicas y no las reduce automáticamente, para conservar el orden de generación.

Las pruebas automáticas incluyen ejemplos con resultados conocidos, dimensiones incompatibles, límites de recursos y desplazamientos que atraviesan posiciones de 64 bits. En ejemplos pequeños se comprueba además que YYC y BT recuperen los testores típicos esperados.

Construcción de la matriz base del experimento

Los datos de partida de este proyecto son:

Matriz	Filas	Columnas	$\lvert\Psi^*\rvert$	Tamaños de los testores típicos
A	6	7	8	5 de tamaño 2 y 3 de tamaño 3
B	8	7	6	4 de tamaño 3 y 2 de tamaño 4

Como A y B tienen distinta cantidad de filas, no se aplica $\varphi(A, B)$ directamente. Siguiendo las Tablas 5 y 6 del artículo, primero se construye

$$C = \theta(A, B), \qquad \text{filas}(C) = 6 \cdot 8 = 48, \qquad \text{columnas}(C) = 7 + 7 = 14.$$

Las primeras ocho filas son $a_1 \mid b_1, \ldots, a_1 \mid b_8$; las siguientes ocho corresponden a a_2, y así sucesivamente. Por ejemplo, la primera fila se obtiene de

$$a_1 = (1, 0, 1, 0, 1, 0, 1), \qquad b_1 = (1, 0, 1, 0, 0, 0, 0),$$

y queda

$$c_1 = (1, 0, 1, 0, 1, 0, 1 \mid 1, 0, 1, 0, 0, 0, 0).$$

El bloque de A ocupa $x_1, \ldots, x_7$ y el de B ocupa $x_8, \ldots, x_{14}$. Esta separación es importante al contar los testores típicos.

Cálculo previo del número de testores típicos

Las proposiciones del artículo permiten conocer la respuesta esperada antes de resolver las matrices grandes. En $\theta(U, V)$, los testores típicos son los de U o los de V, cada grupo dentro de su propio bloque de columnas. Si un conjunto no cubriera ni U ni V, existiría una pareja de filas cuya concatenación tampoco quedaría cubierta. Por tanto:

$$|\Psi^*(\theta(U, V))| = |\Psi^*(U)| + |\Psi^*(V)|, \qquad |\Psi^*(C)| = 8 + 6 = 14.$$

De los 14 testores de C, cinco tienen tamaño 2, siete tienen tamaño 3 y dos tienen tamaño 4. Los siete de tamaño 3 provienen de los tres de A y los cuatro de B.

En $\varphi^N(C)$ cada columna original aparece en N copias idénticas. Un testor típico de tamaño k origina N^k selecciones: se elige una copia para cada una de sus k características. No se necesitan dos copias de la misma característica, porque una sería redundante. En consecuencia:

$$|\Psi^*(\varphi^N(C))| = \sum_{T \in \Psi^*(C)} N^{|T|} = 5N^2 + 7N^3 + 2N^4.$$

Por ejemplo, para $N = 2$ se esperan $5 \cdot 4 + 7 \cdot 8 + 2 \cdot 16 = 108$ testores típicos.

En $\gamma(U, V)$ se deben cubrir ambos bloques, de modo que cada testor típico resulta de combinar uno de U con uno de V, desplazando las columnas del segundo bloque:

$$|\Psi^*(\gamma(U, V))| = |\Psi^*(U)|\,|\Psi^*(V)|.$$

Al repetir C en N bloques diagonales, hay 14 elecciones por bloque y las combinaciones son independientes:

$$|\Psi^*(\gamma^N(C))| = 14^N.$$

Por ejemplo, $\gamma^2(C)$ debe tener $14^2 = 196$ testores típicos. Estos conteos son resultados teóricos utilizados para validar las ejecuciones, no tiempos de los algoritmos.

Matrices generadas para las Tablas 5 y 6

Para reproducir la familia de la Tabla 5 se generaron $\varphi^N(C)$ con $N = 1, 2, 3, 4, 5$. Para la Tabla 6 se generaron $\gamma^N(C)$ con $N = 1, 2, 3, 4$. El caso $N = 1$ es C en ambas familias, por lo que sólo se necesitan ocho archivos distintos.

N	Matriz	Filas	Columnas	$\lvert\Psi^*\rvert$ teórico
1	$\varphi^1(C)$	48	14	14
2	$\varphi^2(C)$	48	28	108
3	$\varphi^3(C)$	48	42	396
4	$\varphi^4(C)$	48	56	1040
5	$\varphi^5(C)$	48	70	2250

Tabla 8: Familia de la Tabla 5 del artículo, adaptada a nuestras matrices A y B. Se mantienen 48 filas y aumentan las columnas.

N	Matriz	Filas	Columnas	$\lvert\Psi^*\rvert$ teórico
1	$\gamma^1(C)$	48	14	14
2	$\gamma^2(C)$	96	28	196
3	$\gamma^3(C)$	144	42	2744
4	$\gamma^4(C)$	192	56	38416

Tabla 9: Familia de la Tabla 6 del artículo, adaptada a nuestras matrices A y B. Crecen las dimensiones y el número de testores típicos aumenta exponencialmente.

En la primera familia, las dimensiones son $48 \times 14N$. En la segunda son $48N \times 14N$. Todas las matrices generadas resultaron básicas: la rutina de reducción no eliminó filas nulas, repetidas ni superfilas. Se mantuvo su orden de generación; las copias ordenadas para comparar tiempos se tratan por separado.

Cómo reproducir la generación

Desde la carpeta principal del proyecto, después de compilar en Release, se puede ejecutar en PowerShell:

```
.\scripts\generar_ejercicio_5_1.ps1
```

El script utiliza los archivos `data/matriz_A.txt` y `data/matriz_B.txt`. Guarda una nueva serie en `data/ej5_1/` y las tablas y comprobaciones en `docs/resultados/ej5_1/`. Si esas carpetas ya existen, se debe indicar otra salida o añadir `-Forzar` para reemplazar esa generación. Los archivos originales empleados en las mediciones manuales permanecen en `data/`.

También se puede generar cada operación individualmente:

```
$exe = ".\build-review\Release\testores.exe"
& $exe theta --entrada-a data/matriz_A.txt `
    --entrada-b data/matriz_B.txt `
    --salida data/ejemplo_theta.txt
& $exe phi --entrada data/ejemplo_theta.txt `
    --copias 2 --salida data/ejemplo_phi2.txt
& $exe gamma --entrada data/ejemplo_theta.txt `
    --copias 2 --salida data/ejemplo_gamma2.txt
```

El script comprueba todas las filas generadas contra las concatenaciones y bloques esperados, verifica que las matrices sean básicas y registra sus dimensiones y hashes. Para calcular los conteos teóricos obtiene primero con YYC los testores de A y B, que son matrices pequeñas. No ejecuta BT sobre las matrices grandes ni presenta sus conteos analíticos como si fueran resultados de una ejecución larga.

Con ello queda preparada la entrada del experimento: las matrices se generan de forma reproducible y sus propiedades esperadas están determinadas. La comparación de tiempos de YYC y BT, con y sin ordenar las filas, constituye el punto 5.2.

7.2. Ejercicio 5.2: resultados experimentales de YYC y BT

Versión de la serie manual. Estos tiempos se conservaron de la versión anterior a la integración de las correcciones de BT (revisión `66804cb`). Esa versión elegía la primera fila no cubierta para el salto S1. La implementación corregida elige, entre todas las filas no cubiertas, aquella cuyo último uno está más a la izquierda. Las mediciones BT de estas tablas son históricas y no caracterizan los tiempos de la implementación corregida. Su serie actualizada en `resultados/experimentos.csv` se mantiene separada.

Se procesaron las matrices del punto 5.1 con YYC y BT, primero con las filas en su orden de generación y después ordenadas de menor a mayor cantidad de unos. En los empates se conservó el orden relativo original. El ordenamiento se realizó antes de iniciar la medición y no forma parte del tiempo del algoritmo.

Las Tablas 10 y 11 adaptan las Tablas 5 y 6 de [2]: conservan las primeras cinco columnas y añaden los cuatro tiempos solicitados. Se utiliza $C = \theta(A, B)$; el caso $N = 1$ es la misma matriz en ambas familias y comparte sus mediciones. Los conteos de testores corresponden a las propiedades calculadas en 5.1, comprobadas con YYC.

Se registró una ejecución manual por caso, utilizando el tiempo reportado por el programa, sin traza. En YYC se acumulan los intervalos de procesamiento de las filas; en BT se mide la resolución, excluyendo la verificación posterior por minimalidad. La impresión y exportación quedan fuera de la medición. Los valores están expresados en milisegundos; los casos sin resultado final se identifican como *falta ejecutar* y no se completan con estimaciones. El análisis de estos resultados se presenta en 5.3.

					YYC (ms)		BT (ms)	
N	Matriz	Filas	Cols.	$\lvert\Psi^*\rvert$	Original	Ascendente	Original	Ascendente
1	C	48	14	14	0.2622	0.1681	0.7632	1.027
2	$\varphi^2(C)$	48	28	108	13.448	5.122	1 169	1 124
3	$\varphi^3(C)$	48	42	396	231.593	57.589	806 341	3 055 669
4	$\varphi^4(C)$	48	56	1040	1 847	367.384	falta ejecutar	falta ejecutar
5	$\varphi^5(C)$	48	70	2250	14 095	2 192	falta ejecutar	falta ejecutar

Tabla 10: Resultados manuales de YYC y BT sobre $\varphi^N(C)$, en orden original y ascendente. Todos los tiempos están en ms.

					YYC (ms)		BT (ms)	
N	Matriz	Filas	Cols.	$\lvert\Psi^*\rvert$	Original	Ascendente	Original	Ascendente
1	C	48	14	14	0.2622	0.1681	0.7632	1.027
2	$\gamma^2(C)$	96	28	196	34.407	11.666	31 476	25 520
3	$\gamma^3(C)$	144	42	2744	280.705	337.262	69 283 000*	falta ejecutar
4	$\gamma^4(C)$	192	56	38416	10 640	15 790	falta ejecutar	falta ejecutar

Tabla 11: Resultados manuales de YYC y BT sobre $\gamma^N(C)$, en orden original y ascendente. Todos los tiempos están en ms.

El asterisco de BT original sobre $\gamma^3(C)$ identifica el registro de 19 h 14 min 43 s, equivalente a 69 283 000 ms. No se verificó la equivalencia de sus condiciones con las de la serie local; por ello se conserva como registro adicional, sin utilizarlo para calcular ventajas entre algoritmos.

7.3. Ejercicio 5.3: análisis del desempeño de YYC y BT

El objetivo de este apartado es comparar los tiempos de YYC y BT y observar cómo cambia su desempeño al ordenar las filas por cantidad ascendente de unos. Se utilizan las mediciones manuales registradas durante las ejecuciones; Los tiempos de BT corresponden a la versión histórica identificada en 5.2, no a la implementación corregida integrada posteriormente. Tampoco se mezclan con la serie automática previa que tenía un límite de 60 segundos por proceso.

Condiciones de las mediciones

La serie local se realizó en un equipo con procesador Intel Core Ultra 7 155H, 16 GB de RAM y Windows de 64 bits. La frecuencia de 1.40 GHz mostrada en la información del sistema es un dato de identificación del equipo, no una medición de la frecuencia efectiva durante cada ejecución. Estas especificaciones corresponden a esta serie del ejercicio 5 y son distintas del entorno descrito para los ejercicios 1 a 3.

Se registró una ejecución por combinación de matriz, algoritmo y orden de filas. El orden ascendente se obtuvo contando los unos de cada fila; en caso de empate se conservó su orden relativo. El ordenamiento se realizó antes de ejecutar el algoritmo, por lo que su costo no se incluye en los tiempos presentados. Todos los valores de las tablas se expresan en milisegundos: los microsegundos se dividieron para 1000 y los segundos se multiplicaron por 1000.

Se tomó el valor que el programa muestra como tiempo total de ejecución. En YYC corresponde a la suma de los intervalos de procesamiento de las filas; en BT corresponde al intervalo de resolución del algoritmo, sin la verificación posterior por minimalidad. La impresión y exportación de resultados quedan fuera de estos intervalos. Los criterios de medición son, por tanto, los implementados en cada algoritmo y no el tiempo total que permanece abierta la terminal.

También se recibió un registro de BT sobre $\gamma^3(\theta(A,B))$, en orden original, de 19 horas, 14 minutos y 43 segundos, reportado por el programa:

$$19 \cdot 3600 + 14 \cdot 60 + 43 = 69\,283 \text{ s} = 69\,283\,000 \text{ ms}.$$

Este dato se identifica con un asterisco porque no se verificó la equivalencia de sus condiciones de ejecución con las de la serie local. Se conserva como registro adicional, sin atribuirlo al equipo local ni utilizarlo para calcular ventajas de un algoritmo sobre otro.

Como se dispone de una observación por caso y no se controlaron de forma documentada la carga del sistema, la temperatura o la posible concurrencia de procesos, las diferencias describen estas ejecuciones. No permiten estimar una variabilidad estadística ni concluir que diferencias pequeñas se repetirán en cualquier sesión.

Resultados registrados

Se utiliza $C = \theta(A,B)$. Las mediciones analizadas son las de las Tablas 10 y 11 del punto 5.2. Los casos sin resultado final conservan el estado *falta ejecutar*; sus proyecciones se presentan separadas más adelante y no se incorporan a las tablas de tiempos medidos.

Comparación entre los algoritmos

En los casos de la serie local donde ambos algoritmos finalizaron, YYC registró un tiempo menor que BT. La diferencia se hace especialmente grande en $\varphi^3(C)$: en orden original, YYC tardó 231.593 ms, mientras BT tardó 806 341 ms, aproximadamente 13 minutos y 26 segundos. La razón entre esos tiempos es cercana a 3482. En orden ascendente, YYC tardó 57.589 ms y BT 3 055 669 ms, aproximadamente 50 minutos y 56 segundos; en esa ejecución, la razón fue cercana a 53 060. Estas razones se refieren a las mediciones registradas y no a constantes de rendimiento de los algoritmos.

La diferencia tiene una explicación en sus estrategias de búsqueda. YYC construye los testores típicos de manera incremental y conserva los conjuntos compatibles de las filas ya procesadas. BT recorre candidatos en orden binario y aplica saltos para omitir bloques que no pueden contener testores típicos. Sin embargo, durante ese recorrido también puede analizar testores que no son típicos. Por ello, tener pocos testores típicos finales no garantiza que el recorrido de BT sea corto.

En la familia $\varphi^N(C)$ se mantienen 48 filas y las columnas pasan de 14 a 70. La cantidad de testores típicos es

$$5N^2 + 7N^3 + 2N^4,$$

pero el espacio de todos los subconjuntos de columnas tiene 2^{14N} elementos, incluido el vacío. Por ejemplo, con 70 columnas existen $2^{70} \approx 1.18 \cdot 10^{21}$ subconjuntos posibles. BT no los visita necesariamente todos: sus saltos reducen la búsqueda. Aun así, esta escala ayuda a comprender por qué algunas entradas pueden requerir mucho trabajo aunque su matriz tenga pocas filas.

En la familia $\gamma^N(C)$ aumentan tanto las filas como las columnas: la matriz tiene $48N$ filas, $14N$ columnas y 14^N testores típicos. Esto incrementa también el volumen de resultados que cualquier algoritmo completo debe producir. Las dos familias permiten estudiar fenómenos distintos, como propone la metodología de [2]. No se puede explicar el tiempo utilizando solamente la cantidad de filas, columnas o testores finales de manera aislada.

Efecto del ordenamiento sobre YYC

En las cuatro matrices $\varphi^2(C), \ldots, \varphi^5(C)$, el orden ascendente produjo un tiempo menor que el original. Para expresar esa diferencia se calcula el factor observado

$$S = \frac{t_{\text{original}}}{t_{\text{ascendente}}}.$$

Los factores fueron aproximadamente 2.63, 4.02, 5.03 y 6.43, respectivamente. Por ejemplo, en $\varphi^5(C)$ el tiempo pasó de 14.095 s a 2.192 s.

Una explicación posible es que las filas con menos unos imponen restricciones más fuertes al inicio y pueden reducir la cantidad de conjuntos intermedios que YYC debe mantener o extender. Esta es una interpretación consistente con su funcionamiento; no se midió en esta serie un perfil completo de operaciones que permita atribuir toda la diferencia a una sola causa.

El efecto no fue uniforme en la familia γ. En $\gamma^2(C)$ el tiempo bajó de 34.407 a 11.666 ms. En cambio, en $\gamma^3(C)$ subió de 280.705 a 337.262 ms, un aumento aproximado del 20.15 %, y en $\gamma^4(C)$ pasó de 10.640 a 15.790 s, un aumento del 48.40 %. Por tanto, ordenar por cantidad de unos no es una regla que siempre acelere YYC. También influye cómo se distribuyen esas entradas entre columnas y cómo las filas de distintos bloques interactúan durante la construcción de los conjuntos intermedios.

Efecto del ordenamiento sobre BT

Los resultados completos de la serie local tampoco muestran una mejora uniforme para BT. En $\varphi^2(C)$ el tiempo pasó de 1.169 a 1.124 s, una diferencia pequeña en una sola observación. En $\varphi^3(C)$ sucedió lo contrario: aumentó de 806.341 a 3055.669 s, aproximadamente 3.79 veces. En $\gamma^2(C)$ disminuyó de 31.476 a 25.520 s. En el caso base C aumentó de 0.7632 a 1.027 ms, aunque su corta duración aconseja no generalizar a partir de esa diferencia aislada.

En la versión histórica de BT usada para estas mediciones, el salto aplicado a un candidato que no es testor depende de la primera fila que queda sin cubrir. Cambiar el orden de las filas puede cambiar esa primera fila y, con ello, la posición que determina el salto y el trabajo de comprobación. Ordenar por número de unos no optimiza directamente ese mecanismo. Esto explica por qué la misma ordenación puede ayudar a un algoritmo y perjudicar al otro. En

particular, en $\varphi^3(C)$ favoreció a YYC y coincidió con un aumento considerable del tiempo de esa versión de BT.

En la implementación corregida, S1 considera todas las filas no cubiertas y utiliza el último uno situado más a la izquierda. Reordenar las mismas filas no cambia ese límite ni la secuencia de candidatos; sí puede variar el costo de las comprobaciones. Los factores de tiempo de la versión histórica no se trasladan a la implementación corregida.

Falta ejecutar

Los siguientes casos no tienen un resultado final registrado en la serie utilizada para este reporte:

- BT original y ascendente sobre $\varphi^4(C)$.
- BT original y ascendente sobre $\varphi^5(C)$.
- BT ascendente sobre $\gamma^3(C)$.
- BT original y ascendente sobre $\gamma^4(C)$.

Su estado es **falta ejecutar**. No se analizan como resultados obtenidos ni se les asigna un tiempo observado.

Estimaciones exploratorias de los casos pendientes

Para describir la escala que podrían alcanzar las ejecuciones se plantea un **escenario de extrapolación geométrica no validado**. El escenario corresponde exclusivamente a la versión histórica de BT. Estas cifras no sustituyen los casos pendientes ni predicen de forma fiable cuándo terminará un proceso que ya está ejecutándose. Tampoco son cotas inferiores, superiores ni intervalos de confianza.

Para cada orden de filas de la familia φ, se toma el factor entre los dos últimos casos completos de BT:

$$r_{\varphi,o} = \frac{806.341}{1.169} \approx 689.77, \qquad r_{\varphi,a} = \frac{3055.669}{1.124} \approx 2718.57.$$

Todos los valores de estas razones están en segundos. Si, como hipótesis, se mantuviera ese factor por cada copia adicional, el modelo sería

$$\widehat{t}_{\varphi,o}(N) = 806.341\, r_{\varphi,o}^{N-3}, \qquad \widehat{t}_{\varphi,a}(N) = 3055.669\, r_{\varphi,a}^{N-3}.$$

Para γ puede construirse únicamente un escenario *condicional*: suponiendo que el registro prolongado sea comparable con el de $\gamma^2(C)$ original, el factor sería

$$r_\gamma = \frac{69\,283}{31.476} \approx 2201.14.$$

Aplicar ese mismo factor al orden ascendente requiere una segunda hipótesis, aún no comprobada. Bajo ambas hipótesis se tendría $\widehat{t}_{\gamma,o}(4) = 69\,283 r_\gamma$, $\widehat{t}_{\gamma,a}(3) = 25.520 r_\gamma$ y $\widehat{t}_{\gamma,a}(4) = 25.520 r_\gamma^2$.

Los valores extremos evidencian la sensibilidad del modelo a sus supuestos. Por ejemplo, la cifra de 715.62 años no significa que una ejecución vaya a durar ese tiempo: resulta de prolongar dos veces una razón muy grande obtenida con sólo dos observaciones. No hay datos suficientes para validar que esa razón permanezca constante. La conclusión defendible es que algunos casos pueden ser imprácticos de completar dentro del plazo disponible, no que se conozca su fecha de finalización. Hasta obtener una ejecución completa, conservan el estado *falta ejecutar*.

La explicación de las duraciones elevadas no debe reducirse a calificar el computador como lento. La implementación de BT recorre sus candidatos de forma secuencial; disponer de varios núcleos no convierte automáticamente ese recorrido en paralelo. Además, un equipo con 16 GB de RAM no elimina el crecimiento combinatorio del trabajo. La potencia del procesador y la carga del sistema pueden modificar el tiempo, pero la estructura de la matriz y la estrategia de búsqueda siguen siendo determinantes.

Matriz	Orden de BT	Proyección del escenario
$\varphi^4(C)$	Original	6.44 días
$\varphi^4(C)$	Ascendente	96.15 días
$\varphi^5(C)$	Original	12.16 años
$\varphi^5(C)$	Ascendente	715.62 años
$\gamma^3(C)$	Ascendente	15.60 horas
$\gamma^4(C)$	Original	4.83 años
$\gamma^4(C)$	Ascendente	3.92 años

Tabla 12: Estimaciones exploratorias, no tiempos medidos ni pronósticos validados. Para expresar años se usaron 365.25 días. Las proyecciones de γ dependen además de una equivalencia de condiciones no verificada.

Conclusiones del análisis

En las comparaciones completas de la serie local, YYC obtuvo menores tiempos que BT. Ordenar las filas por cantidad ascendente de unos benefició a YYC en la familia φ, pero no en todos los casos de γ. En BT se observaron tanto disminuciones como aumentos, incluido el incremento de aproximadamente 3.79 veces en $\varphi^3(C)$. Por ello, el ordenamiento debe evaluarse para cada familia de matrices y no asumirse como una mejora universal.

Los resultados apoyan la utilidad de experimentar con distintas estructuras de matrices, en lugar de concluir el desempeño de un algoritmo a partir de una única entrada. Las conclusiones anteriores se limitan a las ejecuciones registradas; los casos pendientes y las proyecciones no se utilizan para afirmar resultados que todavía no se han obtenido.

Referencias

[1] E. Alba-Cabrera, J. Ibarra-Fiallo, S. Godoy-Calderón y F. Cervantes-Alonso. YYC: A fast performance incremental algorithm for finding typical testors. En *Progress in Pattern Recognition, Image Analysis, Computer Vision, and Applications (CIARP 2014)*, LNCS 8827, pp. 416–423. Springer, 2014.

[2] E. Alba-Cabrera, S. Godoy-Calderón y J. Ibarra-Fiallo. Generating synthetic test matrices as a benchmark for the computational behavior of typical testor-finding algorithms. *Pattern Recognition Letters*, 80:46–51, 2016.

[3] M. Lazo-Cortés, J. Ruiz-Shulcloper y E. Alba-Cabrera. An overview of the evolution of the concept of testor. *Pattern Recognition*, 34(4):753–762, 2001.

[4] *Algoritmo BT para el cálculo de todos los testores típicos*. Documento "Algo BT", material del curso Matemáticas Discretas (sección Teoría de Testores, D2L), Universidad San Francisco de Quito, 202610.

[5] J. Ruiz-Shulcloper, M. A. Bravo y F. L. Águila. Algoritmos BT y TB para el cálculo de todos los tests típicos. *Revista Ciencias Matemáticas*, 6(2), 1982.

[6] A. Pons-Porrata. *Desarrollo de algoritmos para la estructuración dinámica de información y su aplicación a la detección de sucesos*. Tesis doctoral, Universitat Jaume I, Castellón, 2004. Sección 4.2.2.

[7] M. Matsumoto y T. Nishimura. Mersenne Twister: a 623-dimensionally equidistributed uniform pseudo-random number generator. *ACM Transactions on Modeling and Computer Simulation*, 8(1):3–30, 1998.

[8] ISO/IEC 14882:2017. *Programming languages — C++*. International Organization for Standardization, 2017.

[9] I. J. Guevara Romero. *Implementación computacional del algoritmo LEX para el cálculo de testores típicos usando aprendizaje simbólico*. Trabajo de titulación de Ingeniería Matemática, Escuela Politécnica Nacional, Quito, mayo de 2021. Lemas 2.18 y 2.19.

[10] *Proyecto: algoritmos para el cálculo de todos los testores típicos de una matriz básica*. Enunciado del proyecto de Matemáticas Discretas, Universidad San Francisco de Quito, semestre 202610. Ejercicio 4.